VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Belagavi, Karnataka

A Project Report on

SMART TOURIST SAFETY MONITORING AND INCIDENT RESPONSE SYSTEM USING AI, GEO-FENCING AND BLOCKCHAIN-BASED DIGITAL ID

Submitted in partial fulfilment of the requirements for the award of the degree of

[DEGREE] in [DEPARTMENT]

Submitted by

RAKESH G

USN: [USN]

Under the guidance of

[GUIDE NAME]

[Designation], Department of [Department]

DEPARTMENT OF [DEPARTMENT]

ACHARYA INSTITUTE OF TECHNOLOGY

Bangalore – 560107

2026–2027

CERTIFICATE

This is to certify that the project work entitled **"Smart Tourist Safety Monitoring and Incident Response System Using AI, Geo-Fencing and Blockchain-Based Digital ID"** is a bona fide work carried out by **Rakesh G** (USN: [USN]) in partial fulfilment of the requirements for the award of the degree of [DEGREE] in [DEPARTMENT] of Visvesvaraya Technological University, Belagavi, during the academic year 2026–2027. All corrections and suggestions indicated during internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Signature of Guide	**Signature of HOD**	**Signature of Principal**
[Guide name]	[HOD name]	[Principal name]

External Viva

Name of the Examiners	Signature with date
1.	
2.	

DECLARATION

I, **Rakesh G**, student of [DEGREE], Department of [DEPARTMENT], Acharya Institute of Technology, Bangalore, hereby declare that the project work entitled **"Smart Tourist Safety Monitoring and Incident Response System Using AI, Geo-Fencing and Blockchain-Based Digital ID"** has been carried out by me under the guidance of [Guide name] and submitted in partial fulfilment of the requirements for the award of the degree of [DEGREE] during the academic year 2026–2027. The work reported here has not been submitted to any other university or institution for the award of any degree.

Place: Bangalore
Date:

Rakesh G
USN: [USN]

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide, [Guide name], for continuous guidance, encouragement and valuable suggestions throughout this project. I also thank Ratnakirti Roy, co-author of the research paper accepted at ICTCS 2026.

I thank [HOD name], Head of the Department of [Department], and [Principal name], Principal, Acharya Institute of Technology, for providing the facilities needed to carry out this work.

I also thank the faculty and staff of the department, the reviewers of ICTCS 2026 for their constructive comments, and my family and friends for their support.

Rakesh G

ABSTRACT

This project builds a Smart Tourist Safety Monitoring and Incident Response System that combines geo-fencing, artificial intelligence and a blockchain-based digital ID. Tourist-safety platforms that use a cloud large language model (LLM) for risk categorisation report high accuracy, but the cloud call dominates alert latency: in the Smart Tourist Safety Monitoring and Incident Response System (STSMIRS) it accounts for 6.8 s of a 12.7 s mean response time and fails without network coverage. This project designs, implements and evaluates a hybrid edge–cloud alternative. A geofencing layer computes Haversine distances, checks circular and polygon hazard zones, measures deviation from the planned route and raises an event after 15 minutes of inactivity. A lightweight classifier on the phone assigns one of four risk levels, and an escalation policy sends only low-confidence or severe events to the cloud LLM. A permissioned proof-of-authority blockchain run by the tourism department, police and immigration office issues signed digital tourist IDs, supports QR-code verification and revocation, keeps personal data off-chain, and records every alert as a tamper-evident incident log.

Using 20,000 synthetic events that match the published class mix and a cloud stand-in calibrated to the reported 89.6% accuracy, eight on-device models were compared with held-out testing, bootstrap confidence intervals and paired significance tests. A 0.66 KB logistic-regression model running in about 6 µs was selected, and class-balanced training raised its CRITICAL recall from 44.3% to 68.8%. A safety-aware escalation policy attains 86.3% accuracy and 86.4% CRITICAL recall while reducing estimated end-to-end latency by 13.6%; a latency-first setting reduces it by 49.3% at 69.2% accuracy. The blockchain rejected all forged, expired and revoked IDs in validation and detected every attempt to alter a past incident; an equivalent Solidity smart contract passed all 13 of its tests. The integrated system was validated end to end on simulated GPS scenarios, including offline operation, and passed all 130 automated tests, including end-to-end browser tests. It is deployed as a real application: an installable tourist mobile web app with real GPS, on-device AI that works offline and an SOS button; a police dashboard with live alerts and QR-code ID verification; and a server with authentication, a database and a real LLM connection.

Keywords: tourist safety, geo-fencing, edge AI, selective LLM escalation, blockchain, digital identity, smart contract

TABLE OF CONTENTS

1. Introduction....1
1.1 Background....1
1.2 Motivation....1
1.3 Problem Statement....1
1.4 Objectives....2
1.5 Scope of the Project....2
1.6 Publication....3
1.7 Organisation of the Report....3
2. Literature Survey....4
2.1 Tourist Safety and Geofencing....4
2.2 Edge Computing and Collaborative Inference....4
2.3 Selective Classification and Model Cascades....4
2.4 Imbalanced and Cost-Sensitive Learning....4
2.5 Blockchain and Digital Identity....5
2.6 Summary and Research Gap....5
3. System Requirements Specification....6
3.1 Functional Requirements....6
3.2 Non-Functional Requirements....6
3.3 Hardware and Software Requirements....7
4. System Design....8
4.1 System Architecture....8
4.2 Hybrid Edge–Cloud Risk Classification....8
4.3 Module Design....9
4.4 Data Flow....10
4.5 Geofencing Design....11
4.6 Escalation Policy Design....11
4.7 Blockchain and Digital-ID Design....11
4.8 Synthetic Data Design....12
5. Implementation....14
5.1 Development Environment....14
5.2 Geofencing Module....14
5.3 Feature Engineering and Consolidation....14
5.4 Model Training and Selection....15
5.5 Class-Imbalance Handling....15
5.6 ONNX Export and Deployment....15
5.7 Hybrid Escalation and Cloud Model....15

5.8 Integrated System....16
5.9 Blockchain Module....16
5.10 Smart Contract....17
5.11 Deployed Application....17
5.12 Analytics Dashboard....19
6. System Validation and Testing....20
6.1 Testing Strategy....20
6.2 Architecture Validation....20
6.3 End-to-End Execution....20
6.4 Integrated System versus Simulation....21
6.5 Blockchain Validation....21
6.6 Automated Test Suite....21
6.7 Smart-Contract Tests....25
6.8 Real-Application and End-to-End Tests....25
7. Results and Discussion....28
7.1 On-Device Model Comparison....28
7.2 Class Imbalance and the CRITICAL Class....29
7.3 Deployment Benchmark....30
7.4 Hybrid Accuracy–Latency Trade-off....31
7.5 Sensitivity to the Cloud Model....32
7.6 Robustness to Distribution Shift....33
7.7 Blockchain Performance....33
7.8 Discussion....34
7.9 Limitations....34
8. Conclusion and Future Enhancements....35
8.1 Conclusion....35
8.2 Future Enhancements....35
References....37
Appendix A: Application Screenshots....39
Appendix B: How to Run the Project....48

LIST OF FIGURES

Fig. 4.1 Overall architecture of the Smart Tourist Safety Monitoring and Incident Response System....8

Fig. 4.2 Hybrid edge–cloud architecture....8

Fig. 4.3 Demonstration geofence map: planned route (blue), hazard zones (red), safe zone (green), GPS traces of the five scenarios and the events raised by Stage 1....11

Fig. 4.4 Structure of the permissioned blockchain: hash-linked, signed blocks of signed transactions....12

Fig. 4.5 Digital-ID issuance and verification sequence....12

Fig. 5.1 Deployed application: tourist phone app, police dashboard, application server and cloud LLM....18

Fig. 6.1 Tourist app during the end-to-end test: GPS stopped in a hazard zone, verified risk level shown on the phone....26

Fig. 6.2 Police dashboard receiving the same incident live, with the zone, phone numbers, decision path and blockchain transaction....27

Fig. 7.1 Test accuracy and CRITICAL recall of the on-device candidates with 95% bootstrap CIs....29

Fig. 7.2 Grouped permutation importance of the selected model on validation data....29

Fig. 7.3 Per-class recall (bars) and accuracy (black lines) of the selected model under four imbalance strategies....30

Fig. 7.4 Confusion matrix of the deployed (class-balanced) on-device model on the test set.30

Fig. 7.5 Simulated system accuracy versus mean AI-stage latency as θc varies, with the validation-selected operating points....32

Fig. 7.6 Estimated end-to-end alert latency: AI stage (red) plus the fixed STSMIRS pipeline stages....32

Fig. 7.7 Safety-aware operating point: system accuracy and CRITICAL recall versus the cloud model's accuracy on escalated cases....33

Fig. A.1 Tourist app: blockchain digital ID with QR credential....39

Fig. A.2 Tourist app without network: alert decided on the phone and saved for later delivery 40

Fig. A.3 Police dashboard: digital-ID verification....41

Fig. A.4 Police dashboard: blockchain explorer after the end-to-end tests....41

Fig. A.5 Analytics dashboard — live risk check: on-device probabilities, escalation decision, final class, action and latency....42

Fig. A.6 Geofence simulation: night-time stop inside the unlit lake periphery, raised by Stage 1 and escalated to the cloud....43

Fig. A.7 Digital ID page: issuing a blockchain digital ID with its QR credential and on-chain record (hashes only)....44

Fig. A.8 Chain explorer: integrity check and the latest blocks....45

Fig. A.9 Results dashboard....46

Fig. A.10 System validation and automated test results.....47

LIST OF TABLES

Table 2.1 Summary of related work....5
Table 3.1 Functional requirements....6
Table 3.2 Non-functional requirements....6
Table 3.3 Development and deployment environment....7
Table 4.1 System modules....9
Table 5.1 Main REST API endpoints....19
Table 6.1 Architecture validation checks....20
Table 6.2 End-to-end scenario execution (20 tourists per scenario)....20
Table 6.3 Blockchain digital-ID validation....21
Table 6.4 Test results by level....21
Table 6.5 Test cases....22
Table 6.6 Smart-contract test cases....25
Table 6.7 End-to-end browser tests of the real application....25
Table 7.1 On-device candidates: validation macro-F1 and test metrics with 95% bootstrap CIs (test n = 4,000)....28
Table 7.2 Paired significance tests....28
Table 7.3 Imbalance strategies for the selected on-device model, alone (test set; recall per class, %)....29
Table 7.4 ONNX Runtime benchmark, batch size 1, one CPU core....30
Table 7.5 Operating points on the test set (thresholds chosen on validation)....31
Table 7.6 Stress tests with model and θc fixed....33
Table 7.7 Blockchain digital-ID performance (demo network, one CPU core)....33

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CI	Confidence Interval
CRIT	CRITICAL risk class
ECE	Expected Calibration Error
GPS	Global Positioning System
LLM	Large Language Model
LR	Logistic Regression
MAC	Multiply–Accumulate operation
MLP	Multilayer Perceptron
ONNX	Open Neural Network Exchange
pp	percentage points
STSMIRS	Smart Tourist Safety Monitoring and Incident Response System
PoA	Proof of Authority
KYC	Know Your Customer
QR	Quick Response code
SHA-256	Secure Hash Algorithm, 256-bit
Ed25519	Edwards-curve digital signature algorithm
TID	Tourist Identifier
θc	Confidence threshold for escalation

CHAPTER 1
INTRODUCTION

1.1 Background

Tourism is a major part of the economy of many Indian cities, and the safety of visitors who are unfamiliar with local streets, transport and risk areas is a growing concern for city administrations. Smart-city tourist-safety services address this by tracking a visitor's GPS position on their smartphone, detecting unusual behaviour such as prolonged inactivity or deviation from a planned route, and alerting authorities when the risk is high.

The Smart Tourist Safety Monitoring and Incident Response System (STSMIRS) [1] is a representative design, and this major project builds a complete working version of it. It combines blockchain-based digital identity, adaptive geofencing and a two-stage detector. In Stage 1 a deterministic gate (Haversine displacement and idle-time thresholds) flags candidate events; in Stage 2 a cloud large language model (LLM) assigns one of four risk levels: LOW, MODERATE, HIGH or CRITICAL. On 652 real test cases the authors report 89.6% accuracy for the four-level decision and a mean alert response time of 12.7 s.

Two further problems arise in practice. First, the tourist must be identified reliably: authorities need to know who is in trouble and whether the person's registration is genuine, but centrally stored passport or Aadhaar data is a privacy and security risk. Second, incident records must be trustworthy: once an alert has been raised, nobody should be able to quietly change its risk level or delete it afterwards. A permissioned blockchain addresses both: it can hold a tamper-evident digital tourist ID and an audit log of incidents without exposing personal data.

1.2 Motivation

Of the 12.7 s alert time reported for STSMIRS, the cloud LLM call alone takes 6.8 s (53.5%); the remaining 5.9 s is spent on detection (0.8 s), database writes (2.1 s), database updates (1.7 s) and notification (1.3 s). In an emergency every second matters, and a cloud call also fails completely when the tourist has no network coverage. The STSMIRS authors themselves note that this dependency on AI-model API latency could be addressed by on-device inference. This project investigates that suggestion.

1.3 Problem Statement

To design, implement and evaluate a smart tourist-safety system that combines geo-fencing, a blockchain-based digital tourist ID and a hybrid edge–cloud risk-

classification system for tourist-safety monitoring in which a lightweight model on the smartphone classifies each geofence event locally, and only uncertain or severe events are escalated to the cloud LLM, so that alert latency is reduced while accuracy and the detection of CRITICAL events are preserved, and every identity and incident record is tamper-evident.

1.4 Objectives

1. Implement the Stage-1 geofencing layer: Haversine distance, circular and polygon hazard zones, route-deviation measurement and idle detection.
2. Build a reproducible synthetic data generator that matches the published STSMIRS class distribution.
3. Train and compare eight on-device candidate models with proper held-out evaluation, confidence intervals and statistical tests, and select one for deployment.
4. Improve recall of the rare, safety-critical CRITICAL class through class-imbalance handling.
5. Export the selected model to ONNX and benchmark latency, memory and size.
6. Design a tunable escalation policy and quantify the latency–accuracy trade-off by simulation.
7. Build a permissioned blockchain that issues signed digital tourist IDs, supports verification and revocation by authorities, records every incident alert, and keeps personal data off-chain; provide an equivalent Ethereum smart contract.
8. Integrate all stages into one working system, validate the architecture end to end, and test it with unit, integration, system and performance tests.
9. Deploy the system as a real application: an installable tourist mobile web app with real GPS, a police dashboard with live alerts, and a server with authentication, a database and a real LLM connection, all tested end to end in a browser.

1.5 Scope of the Project

The project covers the three pillars of the system: geo-fencing (Stage 1), the AI risk classifier with selective cloud escalation (Stage 2), and the blockchain-based digital ID with its incident audit trail. A Streamlit application stands in for the tourist app and the authority dashboard. Because the STSMIRS data are not public and the production LLM could not be called, all experiments use synthetic data and a calibrated stand-in for the cloud model; the results therefore show relative trade-offs rather than field performance.

1.6 Publication

The AI part of this project has been accepted for oral presentation at ICTCS 2026, Ahmedabad, and for publication in the Springer *Lecture Notes in Networks and Systems* series (Paper ID 492): “Latency-Aware Hybrid Edge–Cloud Risk Classification for Tourist Safety Monitoring: A Simulation Study of Selective LLM Escalation”. The numbers in this report come from the re-implemented code delivered with the project and are close to, but not identical to, those in the paper.

1.7 Organisation of the Report

Chapter 2 reviews related literature on tourist safety, edge inference and blockchain identity. Chapter 3 specifies the system requirements. Chapter 4 presents the system architecture and design. Chapter 5 describes the implementation of each module. Chapter 6 covers system validation and testing. Chapter 7 presents and discusses the results. Chapter 8 concludes the report and outlines future enhancements.

CHAPTER 2
LITERATURE SURVEY

2.1 Tourist Safety and Geofencing

Geofence design has moved from hand-drawn zones to data-driven placement. Sasaki *et al.* [2] use GPS trajectories and a genetic algorithm to size point-of-interest geofences for tourists. Karuppuchamy *et al.* [3] use an LSTM to forecast tourist movement so that geofence alerts can be raised before a boundary is crossed. STSMIRS [1] combines adaptive geofencing with blockchain-based digital identity and LLM-based risk categorisation. Trajectory mining [4] provides the stay-point, deviation and dwell-time features such systems use, while public collections such as GeoLife [5] contain raw GPS traces but no labelled safety incidents.

2.2 Edge Computing and Collaborative Inference

Edge computing moves computation close to the data source to reduce latency and bandwidth and to survive intermittent connectivity [6]. Neurosurgeon [7] partitions a neural network between phone and server according to latency and energy, and BranchyNet [8] lets confident inputs exit early from a network. ONNX Runtime [9] executes exported models efficiently on mobile CPUs. For low-dimensional tabular features, linear models and tree ensembles such as random forests [10], gradient boosting [11] and XGBoost [12] are natural on-device candidates.

2.3 Selective Classification and Model Cascades

Chow [13] formalised the reject option: a classifier may abstain when uncertain, trading error rate against rejection rate. Geifman and El-Yaniv [14] learn a confidence threshold that guarantees a target risk on accepted inputs. Cascades apply a cheap model first and an expensive one only when needed, from the Viola–Jones face detector [15] to FrugalGPT [16], which routes queries to larger LLMs only when a cheaper model's answer is unreliable.

2.4 Imbalanced and Cost-Sensitive Learning

When classes are rare and errors have unequal costs, re-weighting training examples [17] or making minimum-expected-cost decisions [18] are standard remedies. In tourist safety the CRITICAL class is both the rarest (9.2% in STSMIRS) and the most costly to miss.

2.5 Blockchain and Digital Identity

A blockchain is an append-only ledger in which every block stores the hash of the previous block, so changing any past record breaks all later links [21]. Ethereum [22] added smart contracts: programs stored on the chain that enforce rules such as who may register or revoke a record. Public chains rely on costly consensus (proof-of-work or proof-of-stake); government and enterprise systems usually use *permissioned* blockchains such as Hyperledger Fabric [24], where only known organisations run nodes and seal blocks, a model known as proof-of-authority. Transactions inside a block are summarised by a Merkle tree [25], whose root changes if any transaction changes, and each transaction is signed with a digital signature such as Ed25519 [26].

Self-sovereign and blockchain-anchored identity systems [23] keep personal data off the chain and store only hashes and signatures on it, so that a verifier can check that a credential was issued by a trusted authority and has not been revoked, without a central database of identity documents. STSMIRS [1] proposes a blockchain-based digital ID for tourists; this project implements that idea together with an incident audit log.

2.6 Summary and Research Gap

Table 2.1 Summary of related work

Work	Approach	Limitation addressed here
Sasaki et al. [2]	Data-driven geofence sizing (GA)	Does not classify risk
Karuppuchamy et al. [3]	LSTM movement prediction for geofence alerts	Cloud/server inference only
STSMIRS [1]	Geofencing + blockchain ID + cloud LLM risk levels	6.8 s LLM latency, fails offline
Neurosurgeon [7], BranchyNet [8]	Edge–cloud partitioning / early exit	Not applied to safety risk levels
Chow [13], FrugalGPT [16]	Reject option / LLM cascades	Second stage assumed reliable; no severity rule
Nakamoto [21], Fabric [24]	Hash-linked ledgers; permissioned consensus	Not applied to tourist identity
This project	Geo-fencing + on-device AI with safety-aware escalation + blockchain digital ID and incident log	—

None of the tourist-safety works study *where* the risk-classification step should run or quantify the latency–accuracy consequences of moving it. This project applies selective classification and cascade ideas to the Stage-2 decision of STSMIRS, with two safety-specific additions: escalation is also triggered by predicted severity, and the cloud model is treated as imperfect on the hard cases it receives.

CHAPTER 3
SYSTEM REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

Table 3.1 Functional requirements

ID	Requirement
FR1	Process a stream of GPS fixes per tourist and compute Haversine distances.
FR2	Detect whether a position lies inside any circular or polygon geofence and whether that zone is a hazard.
FR3	Compute the distance of the tourist from the planned route.
FR4	Raise exactly one candidate event when the tourist is stationary for 15 minutes or more.
FR5	Convert each event into the five on-device features.
FR6	Classify the event on the device into LOW, MODERATE, HIGH or CRITICAL with a confidence.
FR7	Escalate to the cloud when confidence < θc, or (safety-aware policy) when the local class is HIGH or CRITICAL.
FR8	If the cloud is unreachable, keep the local decision and queue the escalation.
FR9	Produce an alert record with final class, recommended action, reason and stage timings.
FR10	Allow θc and the policy to be changed without retraining.
FR11	Register a tourist and issue a digital ID credential signed by an authority, valid for the trip period.
FR12	Store only hashes of personal data on the blockchain; keep readable KYC data off-chain with the issuer.
FR13	Let an authority verify a scanned credential: on-chain record, issuer signature, revocation status and validity dates.
FR14	Let an authority revoke a digital ID.
FR15	Record every incident alert on the blockchain and detect any later modification of the chain.
FR16	Let a tourist raise an SOS that reaches the police as a CRITICAL incident.
FR17	Push every new incident to logged-in police officers in real time and let them acknowledge and resolve it.
FR18	Keep alerts raised without network on the phone and deliver them automatically when the network returns.
FR19	Let police add or remove hazard zones on a map; tourist phones pick up the change automatically.

3.2 Non-Functional Requirements

Table 3.2 Non-functional requirements

ID	Attribute	Requirement
NFR1	Latency	On-device inference below 1 ms; end-to-end latency lower than the cloud-only baseline (12.7 s).
NFR2	Accuracy	System accuracy within 3 percentage points of cloud-only; CRITICAL recall not lower than cloud-only.

ID	Attribute	Requirement
NFR3	Footprint	Deployed model below 2 KB and negligible memory, suitable for a Flutter app.
NFR4	Availability	A decision is always produced, even without network connectivity.
NFR5	Reproducibility	Every number regenerated by one command with fixed random seeds.
NFR6	Maintainability	All constants in one configuration file; modular source code; automated tests.
NFR7	Privacy	No name, document number or phone number on the blockchain.
NFR8	Integrity	Any change to a past block or transaction is detected by the integrity check.
NFR9	Access control	Only authorised authority nodes can write to the blockchain.

3.3 Hardware and Software Requirements

Table 3.3 Development and deployment environment

Item	Specification
Processor	Any 64-bit x86 or ARM CPU (benchmarks: one core of an Intel Xeon server CPU)
Memory	4 GB RAM minimum, 8 GB recommended for the full study
Operating system	Windows 10/11, macOS or Linux
Language	Python 3.10 or later
Libraries	NumPy, pandas, SciPy, scikit-learn, skl2onnx, ONNX Runtime, Matplotlib, psutil
Blockchain	cryptography (Ed25519, SHA-256), qrcode; Solidity 0.8, Node.js, solc, Ganache, ethers.js
Application server	FastAPI, Uvicorn, SQLite, WebSocket, httpx (LLM APIs: Anthropic, OpenAI, Gemini)
Web / mobile front end	HTML5, JavaScript, Leaflet + OpenStreetMap, Geolocation API, Service Worker (PWA), html5-qrcode
Analytics dashboard	Streamlit, Folium
Testing	pytest, pytest-html, Playwright (Chromium), Node.js
Target devices	Any Android / iOS smartphone browser (installable app); police dashboard on any laptop browser

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Fig. 4.1 shows the complete system. The tourist app registers the tourist with the identity service, which writes a REGISTER_ID transaction to the permissioned blockchain and returns a signed digital-ID credential (QR code). While the tourist travels, GPS fixes pass through the geo-fencing layer (Stage 1); every candidate event is classified on the phone (Stage 2a) and, when the escalation policy requires it, verified by the cloud LLM (Stage 2b). The alert dispatcher maps the final risk level to an action, notifies the authorities and records an INCIDENT transaction on the blockchain. Police officers verify a tourist's digital ID by scanning the QR code.

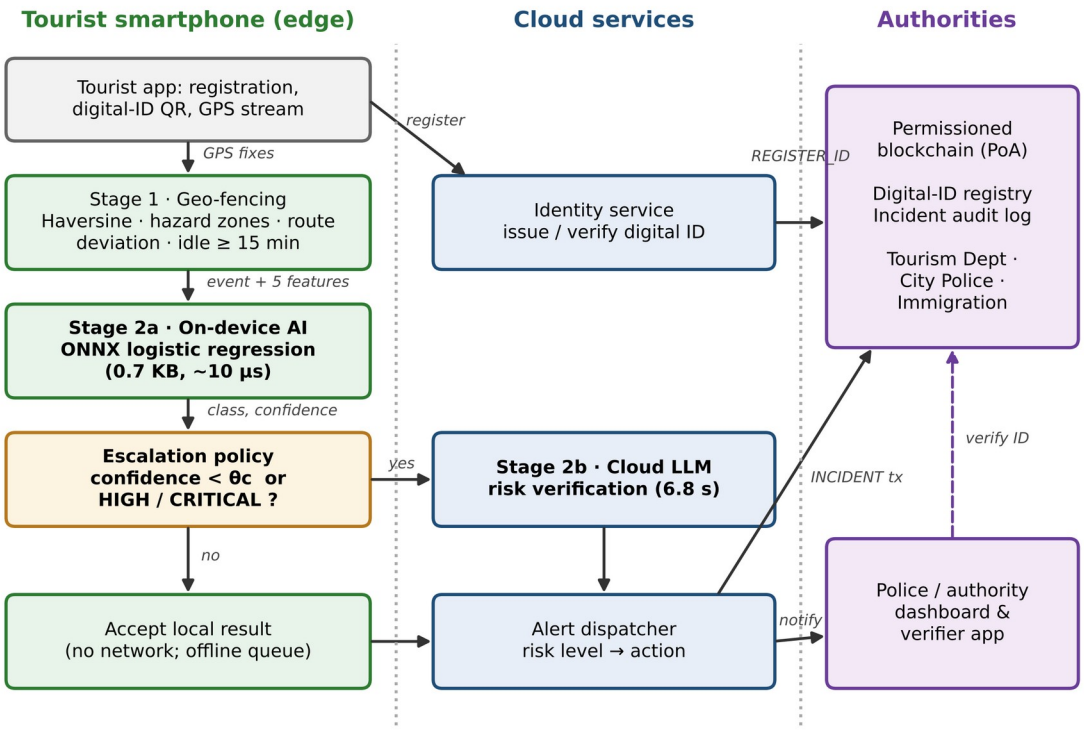


Fig. 4.1 Overall architecture of the Smart Tourist Safety Monitoring and Incident Response System.

4.2 Hybrid Edge–Cloud Risk Classification

Fig. 4.2 shows the hybrid architecture of the AI part in more detail. GPS collection, the Stage-1 gate, identity and the dashboard follow STSMIRS [1]. For every flagged event a lightweight classifier on the phone (Stage 2a) outputs class probabilities; its prediction and confidence c go to the escalation policy. Accepted events are finalised locally without a network round trip; escalated events go to the cloud LLM (Stage 2b). The

dashed loop, in which cloud verdicts become labels for periodic retraining, is part of the design but not evaluated.

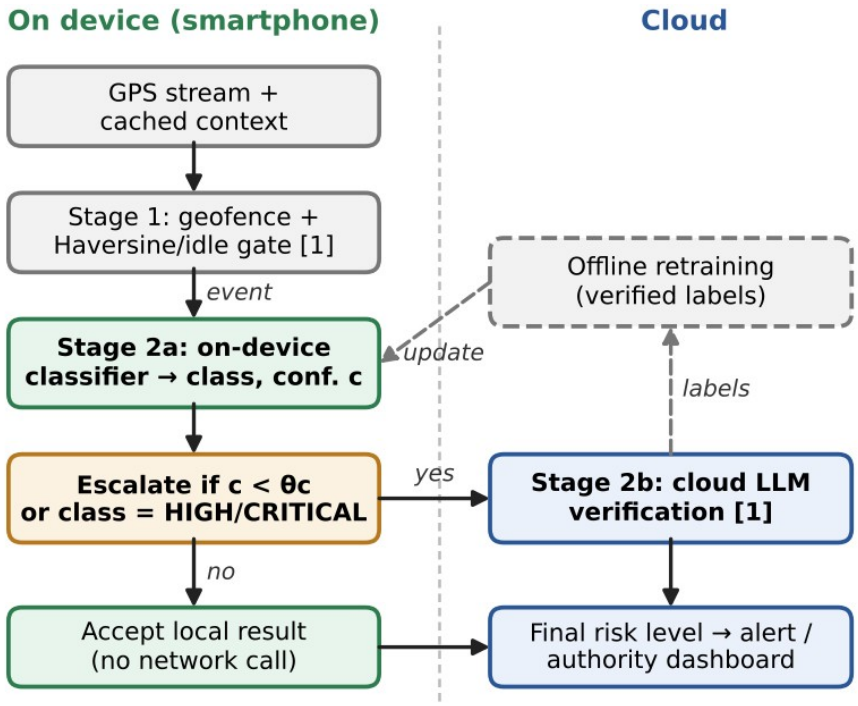


Fig. 4.2 Hybrid edge–cloud architecture. Grey: components unchanged from STSMIRS; green: on-device path; amber: escalation policy; blue: cloud path.

4.3 Module Design

Table 4.1 System modules

Module	Source file	Responsibility
Configuration	src/config.py	All constants: class mix, generator, latency, thresholds
Geofencing (Stage 1)	src/geofence.py	Haversine, zones, route distance, idle gate, event features
Trajectory simulator	src/trajectory.py	Bangalore route, hazard zones, five tourist scenarios
Data generator	src/generator.py	Synthetic post-gate events and feature builder
Models	src/models.py	Eight candidate models, grids, rule baseline
Statistics	src/stats.py	Metrics, bootstrap CIs, McNemar, 5×2 cv F-test, ECE, cost rule
Hybrid simulation	src/hybrid.py	Cloud proxy, escalation policies, latency model
Integrated system	src/system.py	ID check → geofence → ONNX → policy → cloud → alert → blockchain
Blockchain	src/blockchain.py	Signed transactions, Merkle roots, PoA blocks, digital-ID registry, incident log
Demo network	src/id_demo.py	Builds and loads the demo blockchain with 200 demo tourists
Smart contract	contracts/ TouristDigitalID.sol	Ethereum version of the ID registry and incident log

Module	Source file	Responsibility
Flutter model	flutter/risk_classifier.dart	The trained model embedded in the mobile app
Benchmark	src/bench_onnx.py	ONNX Runtime latency and memory measurement
Pipeline	run_all.py, make_figures.py	End-to-end study, tables and figures
Validation	validate_system.py	Architecture validation and execution
Testing	tests/, run_tests.py	Unit, integration, system and performance tests
Application server	server/*.py, run_server.py	REST API, live feed, auth, database, LLM connector, HTTPS
Tourist app	web/tourist/	Installable mobile web app: ID, GPS, on-device AI, offline queue, SOS
On-device engine	web/static/geo.js	Geofence gate and AI model in JavaScript
Police dashboard	web/police/	Live map, incidents, ID verification, zones, blockchain
Analytics dashboard	app/streamlit_app.py	Research results and simulations

4.4 Data Flow

The processing of one tourist proceeds as follows:

1. The phone produces one GPS fix per minute.
2. The geofence gate compares each fix with the previous anchor point; if the tourist stays within 30 m, the idle time accumulates.
3. After 15 minutes idle, the gate creates a candidate event containing the distance from the planned route, the idle duration, the hour of day and the hazard flag of any containing zone.
4. The event is converted into five features: log displacement, log idle time, sine and cosine of the hour, and the hazard flag.
5. The ONNX model returns four class probabilities; the largest gives the local class and its confidence.
6. The escalation policy decides whether to accept the local result or call the cloud LLM.
7. The final class is mapped to an action (log, check-in, notify police, dispatch responders) and an alert record with timings is produced.
8. The alert is anchored on the blockchain as an INCIDENT transaction linked to the tourist’s digital ID.

4.5 Geofencing Design

Distances are computed with the Haversine great-circle formula. Hazard and safe areas are modelled as circular zones (centre and radius) or polygon zones (ray-casting point-in-polygon test). The distance from the planned route is the minimum distance to any segment of the route polyline, computed in a local equirectangular projection that is accurate to well under 1% over city distances. A tourist is stationary while every new fix stays within 30 m of the anchor fix; one event is raised per idle episode when the idle time reaches 15 minutes. The demonstration map is a walking route through central Bangalore with three hazard zones and one safe zone.

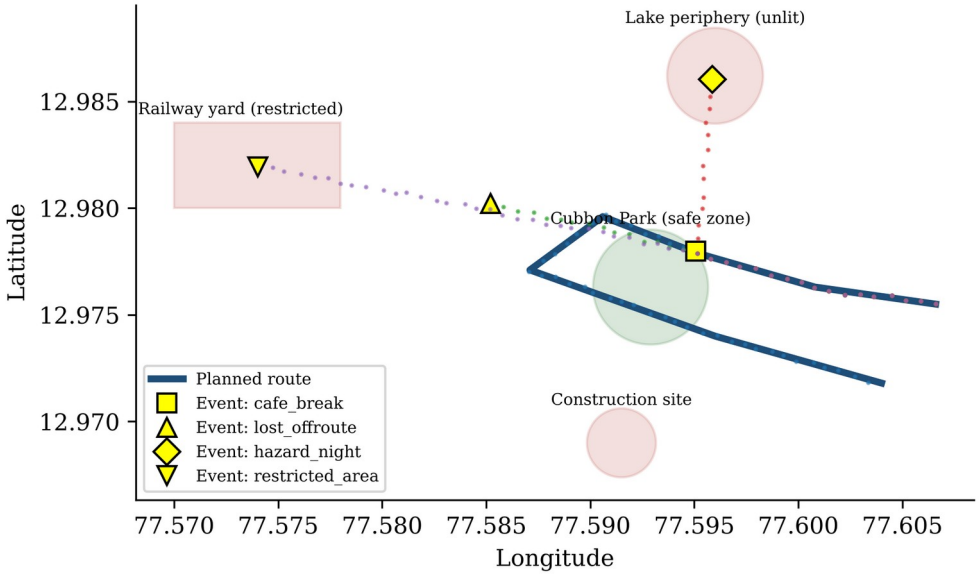


Fig. 4.3 Demonstration geofence map: planned route (blue), hazard zones (red), safe zone (green), GPS traces of the five scenarios and the events raised by Stage 1.

4.6 Escalation Policy Design

Two policies are provided. The *confidence-only* policy escalates when $c < \theta_c$. The *safety-aware* policy also escalates every local HIGH or CRITICAL prediction, so that no severe alert is dispatched without cloud verification. θ_c is chosen on validation data and can be changed per deployment zone without retraining. If the cloud is unreachable, the local prediction is final and the escalation request is queued.

4.7 Blockchain and Digital-ID Design

The blockchain is permissioned: only three authority nodes (Tourism Department, City Police and Immigration Office) hold signing keys and may write. Each transaction (REGISTER_ID, REVOKE_ID, INCIDENT or VERIFY_LOG) is signed by its issuer with Ed25519. Pending transactions are grouped into a block whose header contains the index, timestamp, the SHA-256 hash of the previous block header, the Merkle root of

the transaction hashes and the name of the validator; the validator, chosen round-robin (proof-of-authority), signs the header. Fig. 4.4 shows the structure.

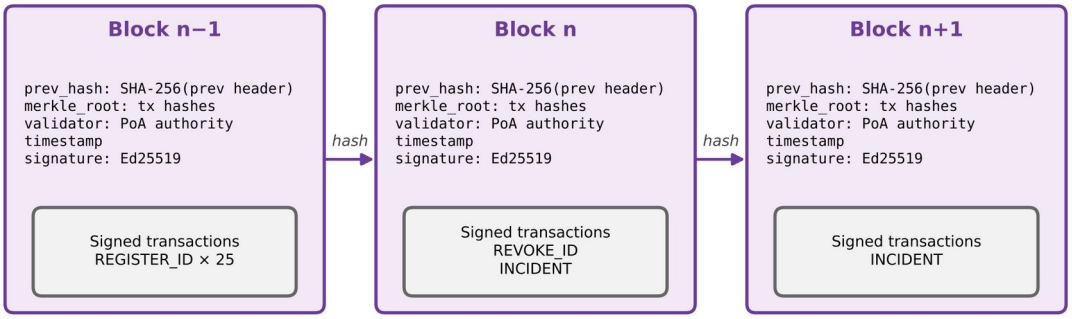


Fig. 4.4 Structure of the permissioned blockchain: hash-linked, signed blocks of signed transactions.

The digital-ID workflow is shown in Fig. 4.5. At registration the issuer collects KYC data, computes a salted hash of it, and derives the tourist ID as a keyed hash of the document number and trip start date, so the same document cannot receive two IDs for one trip. Only the tourist ID, hashes, nationality and validity dates go on-chain. The tourist receives a credential (ID, name, nationality, validity, issuer) signed by the issuer, as a QR code. A verifier checks that the record exists on-chain, that the issuer's signature is valid, that the credential's hash matches the on-chain record, that the ID has not been revoked and that the date is within the validity period.

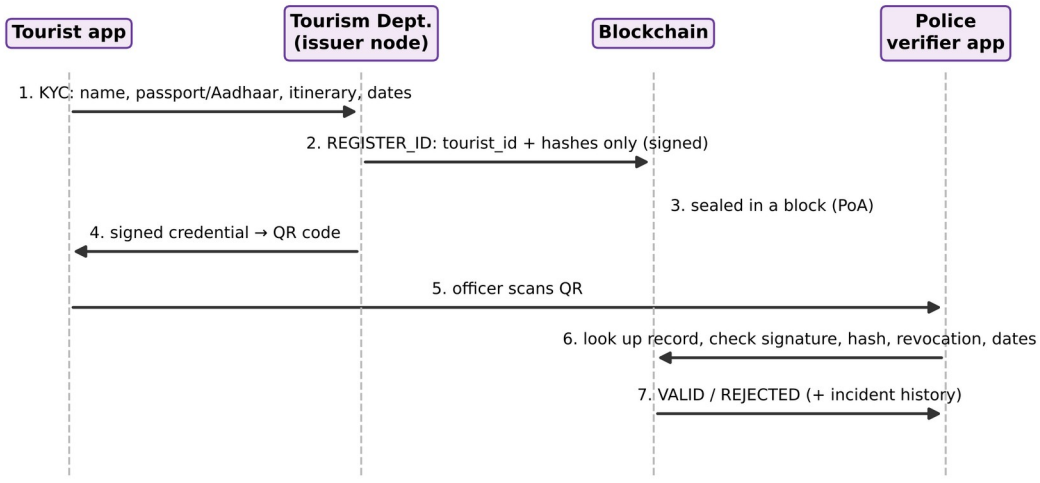


Fig. 4.5 Digital-ID issuance and verification sequence.

4.8 Synthetic Data Design

Because STSMIRS data are not public, a generator creates post-gate events. Each event has a true displacement (log-normal), an idle duration of at least 15 minutes (shifted gamma), an hour of day (uniform), a hazard-zone indicator (25%) and two variables that

only the cloud can observe: a context score and a nearby-incident flag (10%). A latent risk score combines these variables with a smooth night-time profile and noise, and is cut at quantiles that reproduce the STSMIRS class mix (LOW 17.1%, MODERATE 47.0%, HIGH 26.7%, CRITICAL 9.2%). The phone observes degraded inputs: displacement with GPS error and a 20 m floor, and a cached hazard flag that is wrong 15% of the time.

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment

The system is implemented in Python 3 using scikit-learn for model training, skl2onnx and ONNX Runtime for deployment and benchmarking, NumPy, pandas and SciPy for data and statistics, Matplotlib for figures, FastAPI for the application server, plain HTML5 and JavaScript with Leaflet for the tourist and police apps, Streamlit for the analytics dashboard, and pytest and Playwright for testing. All randomness uses fixed seeds, so every number in this report is reproduced by running the pipeline.

5.2 Geofencing Module

The Haversine distance and the Stage-1 gate are the core of the geofencing module:

```
def haversine_m(lat1, lon1, lat2, lon2):
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp, dl = p2 - p1, math.radians(lon2 - lon1)
    a = math.sin(dp/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(dl/2)**2
    return 2 * EARTH_RADIUS_M * math.asin(math.sqrt(a))

class GeofenceGate:
    def update(self, fix):
        if self._anchor is None or haversine_m(*self._anchor.point, *fix.point)
> 30:
            self._anchor, self._fired = fix, False        # tourist moved: new
episode
            return None
        idle = (fix.t - self._anchor.t).total_seconds() / 60
        if idle >= 15 and not self._fired:                 # one event per idle
episode
            self._fired = True
            return CandidateEvent(...,
displacement_m=distance_to_route_m(fix.point, self.route),
                                  idle_min=idle, hour=...,
hazard_flag=self.hazard_at(fix.point))
```

5.3 Feature Engineering and Consolidation

A single feature definition is shared by the training data, the geofence events and the demo, so the deployed model always receives exactly what it was trained on. In the real application the same model and gate also run in JavaScript on the phone (web/static/geo.js); an automated test checks that the JavaScript and Python versions give identical results. Displacement and idle time are log-transformed because their effect on risk is multiplicative; the hour is encoded as sine and cosine so that 23:00 and 01:00 are close; the hazard flag is 0 or 1. The architecture validation (Chapter 6)

confirms that features produced by the geofence gate equal those of the training data exactly.

```
def features(self):
    return np.array([[math.log(max(self.displacement_m, 20.0)),   # GPS floor
20 m
                      math.log(self.idle_min),
                      math.sin(2*math.pi*self.hour/24),
math.cos(2*math.pi*self.hour/24),
                      float(self.hazard_flag)]], dtype=np.float32)
```

5.4 Model Training and Selection

The 20,000 events are split once, stratified, into training (12,000), validation (4,000) and test (4,000) sets. Hyper-parameters are tuned by stratified 5-fold cross-validation on the training set using macro-F1. Eight candidates are compared: a majority-class predictor, a hand-crafted rule score, multinomial logistic regression, a decision tree, a random forest, gradient boosting, histogram gradient boosting and a multilayer perceptron (32–16 units). The selection rule, fixed before testing, treats families within 1 percentage point of the best validation macro-F1 as equivalent and chooses the one with the smallest ONNX model. The models judged equivalent were Logistic regression and MLP (32-16), and **Logistic regression** was selected.

5.5 Class-Imbalance Handling

Four strategies are compared for the selected model: unweighted training; class-balanced sample weights (inverse class frequency); a minimum-expected-cost decision rule in which missing a CRITICAL event costs ten times a false alarm; and both combined. The class-balanced model is adopted as the deployed model.

5.6 ONNX Export and Deployment

Each model is exported to ONNX with skl2onnx and benchmarked with ONNX Runtime in a fresh process pinned to one CPU core: 300 warm-up calls and 5,000 timed calls, repeated three times. The deployed model is also saved as plain coefficients (deployed_model.json), so it can be embedded in the Flutter application as 24 numbers plus 10 scaling constants without any inference runtime.

5.7 Hybrid Escalation and Cloud Model

The cloud LLM could not be called, so it is represented by a *case-dependent proxy*: it observes the full-information latent risk score, including the context the phone cannot see, plus Gaussian noise whose standard deviation ($\sigma = 0.170$) is calibrated so that its accuracy equals the 89.6% reported for STSMIRS. Its errors therefore fall on the ambiguous cases the phone tends to escalate. Cloud latency is log-normal with mean 6.8

s and a 30% coefficient of variation; the fixed 5.9 s of non-AI stages is added to estimate end-to-end latency.

5.8 Integrated System

The class HybridRiskSystem connects all stages. Its decision function is shown below; every alert records the reason for escalation and the time spent in each stage.

```
proba = self.model.predict_proba(event.features())[0]        # Stage 2a (ONNX)
local, conf = int(proba.argmax()), float(proba.max())
escalate = conf < self.theta or (self.policy == "safety" and local >= 2)
final, by = local, "on-device"
if escalate:
    try:
        final, latency = self.cloud.classify(event, context)   # Stage 2b
        by = "cloud LLM"
    except ConnectionError:                                      # no network
        self.offline_queue.append(event)
        by = "on-device (cloud offline, escalation queued)"
return Alert(final_class=CLASSES[final],
action=SEVERITY_ACTION[CLASSES[final]], ...)
```

5.9 Blockchain Module

The blockchain is implemented in src/blockchain.py using SHA-256 hashing and Ed25519 signatures from the Python cryptography library. The core of block sealing and the integrity check is shown below.

```
def seal_block(self, validator=None):                       # proof-of-authority
    validator = validator or names[self._seal_count % len(names)]   # round
robin
    prev = self.chain[-1]
    blk = Block(prev.index + 1, time.time(), prev.hash,
                merkle_root([t.tx_hash for t in self.pending]), validator,
list(self.pending))
    blk.signature = self.authorities[validator].sign(canonical(blk.header()))
    self.chain.append(blk)

def verify_chain(self):
    for i, blk in enumerate(self.chain[1:], start=1):
        if blk.prev_hash != self.chain[i - 1].hash:
problems.append("link broken")
        if blk.merkle_root != merkle_root([t.tx_hash for t in
blk.transactions]):

problems.append("tx altered")
        if not verify_sig(pk[blk.validator], blk.signature,
canonical(blk.header())):

problems.append("bad block signature")
```

The class DigitalIDRegistry acts as the smart contract: register_tourist issues and records a credential, verify performs the five checks of Section 4.7, revoke adds a

REVOKE_ID transaction, and record_incident anchors an alert by storing its SHA-256 hash together with the tourist ID, time, rounded location and risk level. The integrated system verifies the tourist's credential before processing a GPS stream and anchors every alert it produces.

5.10 Smart Contract

For deployment on an Ethereum-compatible permissioned network (for example Hyperledger Besu or a Polygon chain), the same registry is written in Solidity (contracts/TouristDigitalID.sol). It defines role-based access for authorities, registerTourist, revokeTourist, verifyTourist, recordIncident and query functions, and emits events for every change. It was compiled with solc 0.8.26 and tested on a local Ganache Ethereum chain (Section 6.7).

```
function verifyTourist(bytes32 touristId, bytes32 presentedCredentialHash)
    external view returns (bool valid, string memory reason)
{
    TouristRecord storage r = records[touristId];
    if (!r.exists) return (false, "ID not found");
    if (r.credentialHash != presentedCredentialHash) return (false, "credential
does not match record");
    if (r.revoked) return (false, "ID revoked");
    if (block.timestamp < r.validFrom || block.timestamp > r.validTo) return
(false, "ID not valid today");
    return (true, "valid");
}
```

5.11 Deployed Application

The system is deployed as a real client–server application (Fig. 5.1). One Python server (FastAPI) serves a REST API, a WebSocket live feed and two web applications; it can run on a laptop for a campus deployment or on any cloud server. Data are stored in SQLite, and the blockchain, the authority keys and the database survive restarts.

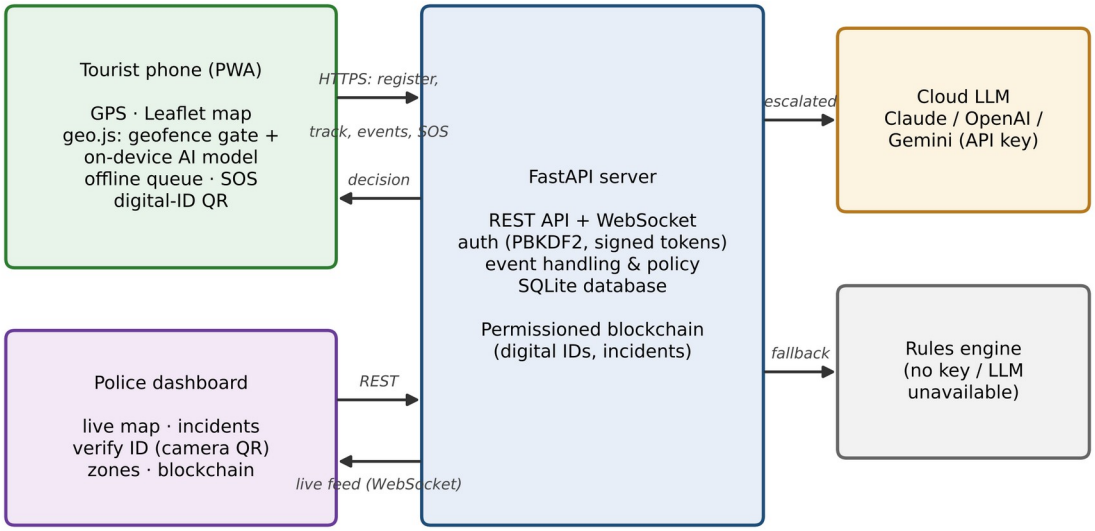


Fig. 5.1 Deployed application: tourist phone app, police dashboard, application server and cloud LLM.

5.11.1 Tourist mobile application

The tourist app opens in any phone browser and can be installed to the home screen as a Progressive Web App; a service worker stores the app so it opens and runs without a network. At registration the tourist enters KYC details and itinerary and receives a blockchain digital ID with its QR code. During the trip the browser's Geolocation API supplies real GPS fixes; the geofence gate and the AI model run on the phone itself (Stage 1 and Stage 2a), so the tourist sees a risk level and recommended action immediately, even offline. Positions and events are sent to the server; when there is no network they are kept on the phone and delivered automatically once the connection returns. A two-tap SOS button sends a CRITICAL incident with the current location.

5.11.2 Police dashboard

Officers log in with a password (stored as a salted PBKDF2 hash) and receive a signed, time-limited token. The dashboard shows a live Leaflet map with every tourist's last position and every open incident. New incidents arrive through a WebSocket within a second, with a sound and a highlighted card showing the risk level, location, zones, phone numbers, how it was decided and the blockchain transaction. Officers acknowledge and resolve incidents, verify a tourist's digital ID by scanning the QR code with the laptop camera, revoke IDs, add hazard zones by clicking on the map, and inspect the blockchain with its integrity check.

5.11.3 Application server and cloud LLM

The server is authoritative: for every event it recomputes the distance from the planned route and the hazard zones from its own data, runs the same model and applies the

escalation policy. Escalated events are sent to a real LLM — Claude (Anthropic), OpenAI or Gemini, selected by API key — with a prompt that adds context the phone does not have: incidents reported within 500 m in the last 24 hours and the tourist's history. The LLM must reply in JSON with a risk level and a reason. If no key is configured, or the call fails or times out, a transparent rules engine decides and the incident records that a fallback was used. Every incident and SOS is anchored on the blockchain. Phones only allow GPS on secure pages, so the launcher can generate a certificate and serve HTTPS on the local Wi-Fi network.

Table 5.1 Main REST API endpoints

Endpoint	Access	Purpose
POST /api/tourists/register	public	Issue a blockchain digital ID
POST /api/track	tourist	Upload GPS fixes
POST /api/events	tourist	Geofence events: model, policy, LLM, blockchain
POST /api/sos	tourist	Emergency alert
POST /api/auth/login	public	Officer login
GET /api/incidents, POST …/ack | resolve	police	Incident workflow
POST /api/verify, POST /api/tourists/{id}/revoke	police	Digital-ID verification and revocation
POST /api/zones, DELETE /api/zones/{id}	police	Hazard-zone management
GET /api/chain	police	Blockchain explorer and integrity check
WS /ws	police	Live feed of positions and incidents

5.12 Analytics Dashboard

A separate Streamlit dashboard presents the research side: a live risk check, the geofence simulation of the five scenarios on a Leaflet map, a Digital ID explorer, the results tables and figures, and the validation and test reports.

CHAPTER 6
SYSTEM VALIDATION AND TESTING

6.1 Testing Strategy

Testing follows four levels: *unit tests* of individual functions (geofencing geometry, generator, statistics, model artefacts); *integration tests* of how the model, policy, cloud proxy and latency model work together; *system tests* that run complete GPS scenarios and the full test set through the integrated system and check the design goals; and *performance tests* of latency, model size and throughput. In addition, an architecture-validation script checks consistency between stages.

6.2 Architecture Validation

Table 6.1 Architecture validation checks

Check	Result	Status
ONNX vs scikit-learn: label agreement on 4,000 test events	100.0%	Pass
ONNX vs scikit-learn: max probability difference	4.2e-7	Pass
Embedded coefficients (Flutter) vs ONNX: max difference	5.0e-7	Pass
Geofence features vs training features: max difference	0	Pass

6.3 End-to-End Execution

Each of the five scenarios was run for 20 simulated tourists (different random GPS traces) through the complete pipeline. Table 6.2 shows the events raised by Stage 1 and the final risk levels.

Table 6.2 End-to-end scenario execution (20 tourists per scenario)

Scenario	Description	Events	LOW	MOD	HIGH	CRIT
normal_walk	Walks the planned route without stopping	0	0	0	0	0
cafe_break	Stops 20 min at a cafe on the route, midday	20	20	0	0	0
lost_offroute	Wanders ~400 m off route and stays 25 min, evening	20	0	20	0	0
hazard_night	Stops 40 min inside the unlit lake periphery at 01:00	20	0	0	0	20
restricted_area	Idle 30 min inside the railway yard at 23:00	20	0	0	0	20

The normal walk raises no event because the tourist never stays still, a short café break is classified as low risk on the device, and the night-time stops inside hazard zones are escalated and classified as severe. With the cloud switched off, the hazard scenario still produced 1 alert decided by the device, and 1 escalation was queued for later, as required by FR8.

6.4 Integrated System versus Simulation

All 4,000 test events were passed one by one through the integrated system (with real ONNX inference and the cloud stand-in). The system reached 87.2% accuracy and 87.2% CRITICAL recall with 74.7% of events escalated, compared with 86.3%, 86.4% and 74.7% in the simulation; the small difference is the random variation of a single cloud run. Measured on-device inference averaged 13 µs (p99 38 µs) inside the full system.

6.5 Blockchain Validation

The validation script registered 200 demo tourists and exercised the full digital-ID life cycle and the incident log.

Table 6.3 Blockchain digital-ID validation

Check	Result	Status
Tourists registered; valid credentials verified	200; 200/200	Pass
Revoked IDs rejected	5/5	Pass
Forged credentials (name changed) rejected	20/20	Pass
Expired IDs rejected	20/20	Pass
Scenario alerts anchored as INCIDENT transactions	16	Pass
Chain integrity check on the real chain	intact	Pass
Past incident rewritten on a copy: tampering detected	detected	Pass
Personal data (names) found in the chain file	none	Pass

When one past incident’s risk level was changed on a copy of the chain, the integrity check reported: “block 10: Merkle root mismatch (a transaction was altered)” and “block 10: invalid signature on transaction d35c12a7c051”.

6.6 Automated Test Suite

The pytest suite contains 130 automated tests; 130 passed and 0 failed. Table 6.4 summarises them by level and Table 6.5 lists every test case.

Table 6.4 Test results by level

Level	Tests	Passed	Failed	Skipped
Unit	61	61	0	0
Integration	40	40	0	0
System	18	18	0	0
End-to-end (browser)	7	7	0	0
Performance	4	4	0	0
Total	**130**	**130**	**0**	**0**

Table 6.5 Test cases

ID	Level	Test case	Result
TC01	System	Demo app page renders (Live risk check)	Pass
TC02	System	Demo app page renders (Geofence simulation)	Pass
TC03	System	Demo app page renders (Digital ID (blockchain))	Pass
TC04	System	Demo app page renders (Results dashboard)	Pass
TC05	System	Demo app page renders (System validation & tests)	Pass
TC06	System	Demo app page renders (About)	Pass
TC07	System	App anchors a geofence alert on the blockchain	Pass
TC08	Unit	SHA-256 matches the standard test vector	Pass
TC09	Unit	Merkle root changes when any transaction changes	Pass
TC10	Unit	Canonical JSON independent of key order	Pass
TC11	Unit	Ed25519 signature verifies; altered data or wrong key fails	Pass
TC12	Unit	Blocks are hash-linked to the genesis block	Pass
TC13	Unit	Unauthorised node cannot write	Pass
TC14	Unit	Validators take turns (proof-of-authority)	Pass
TC15	Unit	No personal data on the chain	Pass
TC16	Integration	Valid digital ID verifies	Pass
TC17	Integration	Forged credential rejected	Pass
TC18	Integration	Credential with extended validity rejected	Pass
TC19	Integration	Expired and not-yet-valid IDs rejected	Pass
TC20	Integration	Revoked ID rejected	Pass
TC21	Integration	ID from another network rejected	Pass
TC22	Integration	Same document cannot get two IDs for one trip	Pass
TC23	Integration	Invalid validity period rejected	Pass
TC24	Integration	Tampered incident detected	Pass
TC25	Integration	Rewritten block breaks the hash link	Pass
TC26	Integration	Chain saved and reloaded intact	Pass
TC27	System	AI alerts anchored on the blockchain	Pass
TC28	System	Demo network (200 tourists) intact	Pass
TC29	Performance	ID verification < 10 ms	Pass
TC30	Unit	Generator reproducible with same seed	Pass
TC31	Unit	Class mix matches STSMIRS ±1 pp	Pass
TC32	Unit	Idle time ≥ 15 min	Pass
TC33	Unit	20 m GPS floor applied	Pass
TC34	Unit	Hazard flag error rate ≈ 15%	Pass
TC35	Unit	Demo features equal training features	Pass
TC36	Unit	Cost rule favours severe class	Pass
TC37	Unit	McNemar p = 1 for identical models	Pass
TC38	Unit	ECE = 0 for perfect predictions	Pass
TC39	Unit	ONNX output equals scikit-learn	Pass
TC40	Unit	Probabilities valid and sum to 1	Pass
TC41	Unit	JSON coefficients reproduce ONNX model	Pass
TC42	Unit	Model beats rule and majority baselines	Pass
TC43	Unit	Balancing raises CRITICAL recall > 15 pp	Pass
TC44	E2E	Landing page	Pass

ID	Level	Test case	Result
TC45	E2E	Real gps tracking alert reaches police	Pass
TC46	E2E	Offline alert is queued and synced	Pass
TC47	E2E	SOS button	Pass
TC48	E2E	Police verify digital ID	Pass
TC49	E2E	Police adds hazard zone on map	Pass
TC50	E2E	Tourist app is installable	Pass
TC51	Unit	Distance from a point to itself is 0	Pass
TC52	Unit	1° latitude ≈ 111.2 km	Pass
TC53	Unit	Bangalore–Chennai ≈ 290 km	Pass
TC54	Unit	Distance is symmetric	Pass
TC55	Unit	Circle zone: 150 m inside, 250 m outside	Pass
TC56	Unit	Polygon point-in-polygon test	Pass
TC57	Unit	Point on route has ~0 m deviation	Pass
TC58	Unit	Point 300 m off route measured correctly	Pass
TC59	Unit	No event before 15 min idle	Pass
TC60	Unit	Exactly one event at 15 min	Pass
TC61	Unit	New episode after moving away	Pass
TC62	Unit	Hazard flag and zone name set	Pass
TC63	Unit	Safe zone does not set hazard flag	Pass
TC64	Unit	Feature vector shape and values	Pass
TC65	Unit	Walking tourist raises no event	Pass
TC66	Unit	Each stopping scenario raises one event (cafe_break)	Pass
TC67	Unit	Each stopping scenario raises one event (lost_offroute)	Pass
TC68	Unit	Each stopping scenario raises one event (hazard_night)	Pass
TC69	Unit	Each stopping scenario raises one event (restricted_area)	Pass
TC70	Unit	Hazard scenarios detect hazard zone	Pass
TC71	Integration	Confidence policy threshold behaviour	Pass
TC72	Integration	Safety policy escalates HIGH/CRITICAL	Pass
TC73	Integration	Cloud proxy calibrated to 89.6%	Pass
TC74	Integration	Cloud latency mean 6.8 s, CV 30%	Pass
TC75	Integration	More escalation gives more latency	Pass
TC76	System	Normal walk: no alert	Pass
TC77	System	Café break: low risk	Pass
TC78	System	Hazard scenarios escalated and severe (hazard_night)	Pass
TC79	System	Hazard scenarios escalated and severe (restricted_area)	Pass
TC80	System	Alert record has all fields and timings	Pass
TC81	System	Offline: local decision kept, escalation queued	Pass
TC82	System	Integrated system reproduces simulation	Pass
TC83	System	Goal G2: accuracy within 3 pp of cloud	Pass
TC84	System	Goal G1: latency below cloud-only	Pass
TC85	Performance	On-device p99 latency < 1 ms	Pass
TC86	Performance	Deployed model < 2 KB	Pass
TC87	Performance	Geofence gate < 1 ms per GPS fix	Pass
TC88	Unit	Haversine route polygon parity	Pass
TC89	Unit	Model parity	Pass

ID	Level	Test case	Result
TC90	Unit	Geofence gate parity on scenarios (normal_walk)	Pass
TC91	Unit	Geofence gate parity on scenarios (cafe_break)	Pass
TC92	Unit	Geofence gate parity on scenarios (lost_offroute)	Pass
TC93	Unit	Geofence gate parity on scenarios (hazard_night)	Pass
TC94	Unit	Geofence gate parity on scenarios (restricted_area)	Pass
TC95	Unit	Time scale accelerates idle	Pass
TC96	Unit	Escalation policy parity	Pass
TC97	Unit	Anthropic request and reply	Pass
TC98	Unit	Openai request and reply	Pass
TC99	Unit	Gemini request and reply	Pass
TC100	Unit	Http error falls back	Pass
TC101	Unit	Unreadable reply falls back	Pass
TC102	Unit	Parse reply ({""risk_level"":""low"")	""reason"":""x""}-LOW]"
TC103	Unit	Parse reply (Risk: HIGH because it is night-HIGH)	Pass
TC104	Unit	Parse reply ({""risk_level"":""EXTREME""}-None")	Pass
TC105	Unit	Prompt contains context	Pass
TC106	Unit	Rules engine	Pass
TC107	Integration	Health and pages	Pass
TC108	Integration	Config has model and zones	Pass
TC109	Integration	Police login and wrong password	Pass
TC110	Integration	Police endpoints require login	Pass
TC111	Integration	Tourist endpoints require token	Pass
TC112	Integration	Admin can add officer	Pass
TC113	Integration	Registration issues blockchain ID	Pass
TC114	Integration	Registration rejects bad input	Pass
TC115	Integration	Duplicate registration same trip blocked	Pass
TC116	Integration	Verify valid forged and revoked	Pass
TC117	Integration	Tracking stores positions	Pass
TC118	Integration	Low risk event decided on device	Pass
TC119	Integration	Hazard event escalated to LLM	Pass
TC120	Integration	LLM failure falls back to rules	Pass
TC121	Integration	Server recomputes features	Pass
TC122	Integration	Offline events synced later and idempotent	Pass
TC123	Integration	SOS is critical	Pass
TC124	Integration	Incident workflow	Pass
TC125	Integration	Every incident anchored on blockchain	Pass
TC126	Integration	Nearby incidents raise context	Pass
TC127	Integration	Police adds zone and it affects risk	Pass
TC128	Integration	Websocket pushes incidents	Pass
TC129	Integration	Websocket rejects without login	Pass
TC130	Integration	Data survives restart	Pass

6.7 Smart-Contract Tests

The Solidity contract was compiled with solc 0.8.26, deployed on an in-memory Ganache Ethereum chain and exercised with ethers.js; 13 of 13 tests passed.

Table 6.6 Smart-contract test cases

ID	Test case	Result
SC01	Admin added police as authority	Pass
SC02	Registered ID verifies	Pass
SC03	Forged credential rejected	Pass
SC04	Duplicate registration reverts	Pass
SC05	Outsider cannot register	Pass
SC06	Invalid period reverts	Pass
SC07	Incident recorded	Pass
SC08	Incident fields stored	Pass
SC09	Incidents indexed by tourist	Pass
SC10	Incident for unknown ID reverts	Pass
SC11	Revoked ID rejected	Pass
SC12	Expired ID rejected	Pass
SC13	Non-admin cannot add authority	Pass

6.8 Real-Application and End-to-End Tests

The real application is covered by 50 further automated tests. API tests run the server in-process: registration, login and access control, tracking, low-risk and escalated events, the LLM connector (with mocked Anthropic, OpenAI and Gemini replies, and a network failure that must fall back to the rules engine), server-side recomputation of features, idempotent offline synchronisation, SOS, the incident workflow, hazard zones, the WebSocket feed, blockchain anchoring and persistence across a restart. Parity tests run the phone's JavaScript engine in Node.js and require identical results to the Python code for distances, zones, the geofence gate on all five scenarios and the model probabilities. Finally, 7 end-to-end tests start the real server and drive two real Chromium browsers: one acts as the tourist's phone with a simulated GPS position that the test moves around, the other as the police dashboard.

Table 6.7 End-to-end browser tests of the real application

ID	Scenario	Result
E2E1	Home page loads and reports the server status	Pass
E2E2	Tourist registers on the phone, GPS moves into a hazard zone and stops; the phone shows the verified risk level and the police dashboard receives the incident live; officer acknowledges it	Pass
E2E3	Phone goes offline, an alert is raised and kept on the phone; when the network returns it is delivered and marked "sent after reconnecting"	Pass
E2E4	SOS pressed twice on the phone arrives at the police as a CRITICAL incident	Pass
E2E5	Officer verifies the tourist's QR credential (VALID); a credential with altered dates is REJECTED	Pass

ID	Scenario	Result
E2E6	Officer adds a hazard zone by clicking the map; the blockchain explorer reports the chain intact	Pass
E2E7	Tourist app has a valid web-app manifest and registers its offline service worker	Pass

Fig. 6.1 shows the two browsers at the end of the main end-to-end test. In the test environment the map tiles are not downloaded, so the map background is blank; on a connected device the OpenStreetMap streets appear.

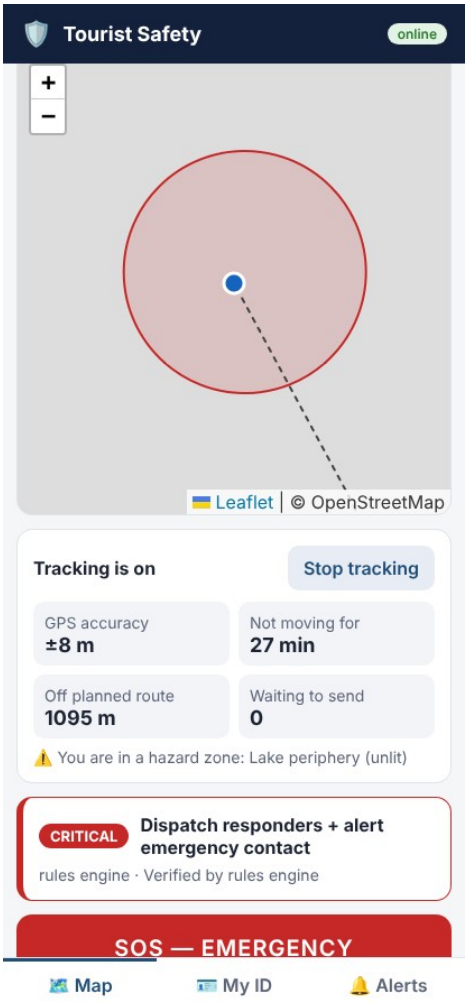


Fig. 6.1 Tourist app during the end-to-end test: GPS stopped in a hazard zone, verified risk level shown on the phone.

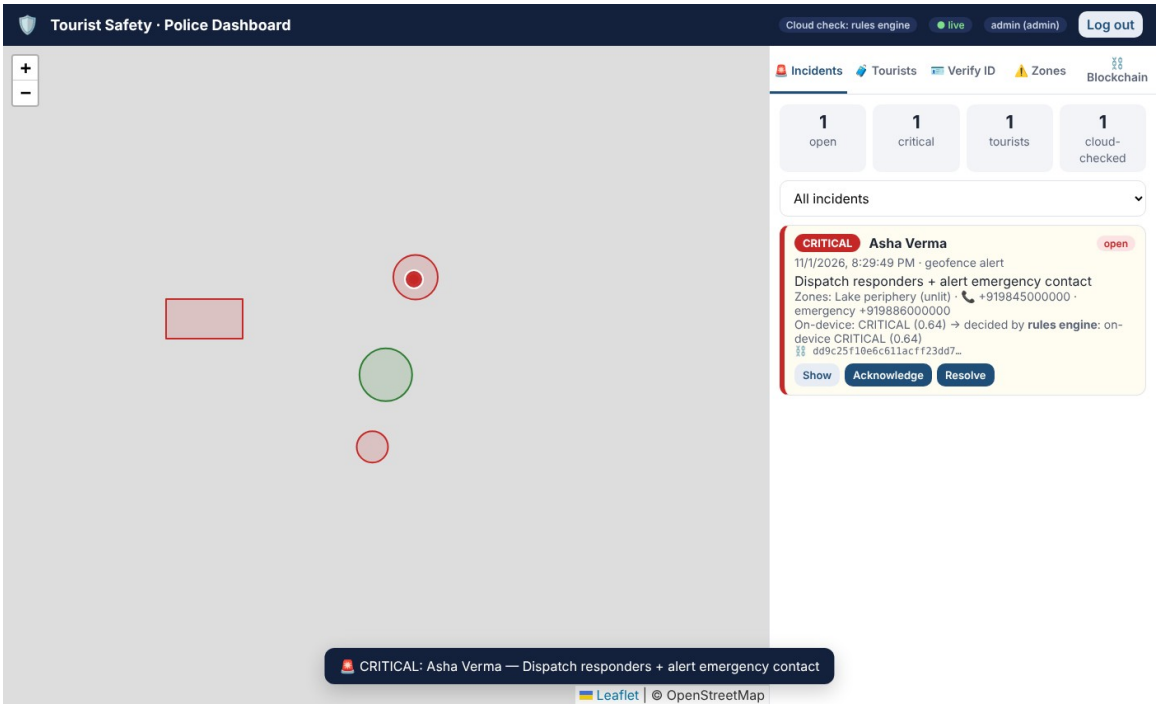


Fig. 6.2 Police dashboard receiving the same incident live, with the zone, phone numbers, decision path and blockchain transaction.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 On-Device Model Comparison

Table 7.1 On-device candidates: validation macro-F1 and test metrics with 95% bootstrap CIs (test n = 4,000)

Model	Val. macro-F1 (%)	Test accuracy (%) [95% CI]	Test macro-F1 (%)	CRITICAL recall (%) [95% CI]	ONNX size (KB)	Latency (µs)
Majority class	16.0	47.0 [47.0, 47.0]	16.0	0.0 [0.0, 0.0]	n/a	–
Rule-based score	64.3	66.9 [65.5, 68.3]	64.3	56.3 [51.1, 61.1]	n/a	–
Logistic regression	**68.7**	**73.3 [71.9, 74.6]**	**68.9**	**44.3 [38.9, 49.5]**	**0.7**	**6**
Decision tree	68.0	71.6 [70.2, 73.0]	67.4	43.8 [38.6, 48.6]	18.8	6
Random forest	67.8	72.0 [70.6, 73.3]	67.1	40.2 [35.0, 45.1]	15319.8	25
Gradient boosting	67.8	72.3 [71.0, 73.6]	67.7	41.0 [35.9, 45.9]	442.2	23
Hist. gradient boosting	67.8	72.7 [71.3, 73.9]	67.4	38.9 [33.7, 43.8]	not exportable	–
MLP (32-16)	69.2	73.0 [71.6, 74.3]	68.8	47.0 [41.8, 51.9]	4.4	11

Latency: ONNX Runtime, one server-CPU core, batch size 1 (not a smartphone).

The six learned models fall within a 1.7 percentage-point band of test accuracy. Paired tests (Table 7.2) show that most differences are not statistically significant, and across 10 independently regenerated datasets logistic regression reached 73.3 ± 0.6% test accuracy (gradient boosting 72.8 ± 0.4%, random forest 72.7 ± 0.3%). Under the selection rule, logistic regression was chosen because its ONNX file is only 0.66 KB. Its confidence is well calibrated (expected calibration error 0.028), which matters because escalation acts on confidence. The rule score is less accurate but has higher CRITICAL recall than any unweighted learned model, a first sign that the default training objective under-serves the rare class.

Table 7.2 Paired significance tests

Comparison	McNemar p (test)	5×2 cv F-test p
Gradient boosting vs Random forest	0.459	0.384
Gradient boosting vs Logistic regression	0.012	0.011
Gradient boosting vs Decision tree	0.155	0.004
MLP (32-16) vs Gradient boosting	0.122	0.015

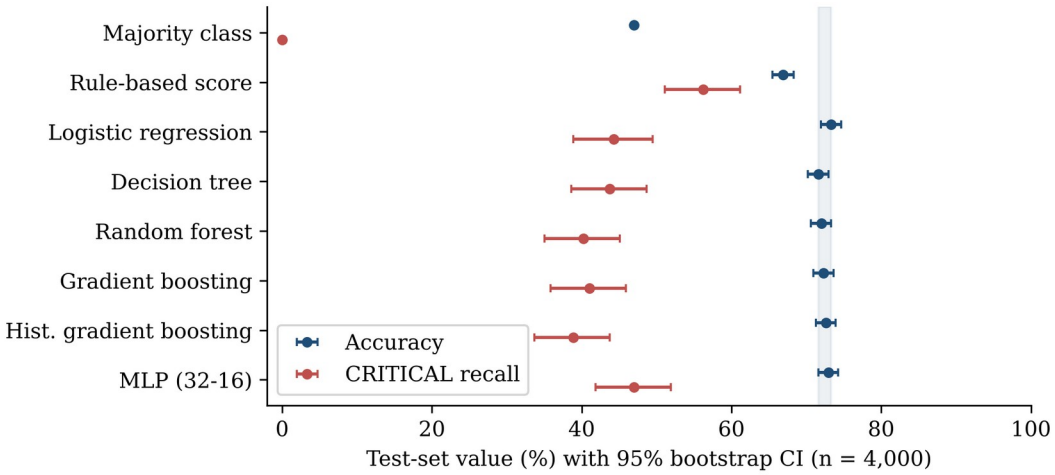


Fig. 7.1 Test accuracy and CRITICAL recall of the on-device candidates with 95% bootstrap CIs. Shaded band: range of the learned models.

Permutation importance (Fig. 7.2) shows that displacement from the planned route dominates (36.0 pp accuracy drop when shuffled), followed by hour of day (12.3 pp) and the hazard flag (6.0 pp). Idle time adds little (0.9 pp) because every event has already passed the idle-based Stage-1 gate.

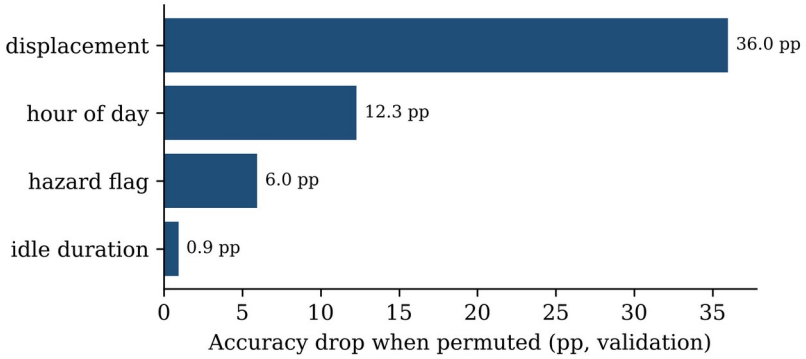


Fig. 7.2 Grouped permutation importance of the selected model on validation data.

7.2 Class Imbalance and the CRITICAL Class

Table 7.3 Imbalance strategies for the selected on-device model, alone (test set; recall per class, %)

Strategy	Accuracy [95% CI]	Macro-F1	LOW	MOD	HIGH	CRITICAL
Unweighted	73.3 [71.9, 74.6]	68.9	68.9	83.9	67.5	44.3
Balanced weights	**65.8 [64.3, 67.2]**	**63.6**	**84.1**	**66.4**	**52.0**	**68.8**
Cost-sensitive	57.0 [55.6, 58.4]	55.1	67.8	63.5	29.4	83.7
Balanced + cost	44.2 [43.0, 45.5]	44.4	82.5	38.9	11.7	94.6

Class-balanced weighting raises CRITICAL recall from 44.3% to 68.8% at a cost of 7.5 pp accuracy. The minimum-cost rules push CRITICAL recall to 83.7% and 94.6% but reduce accuracy to 57.0% and 44.2% by over-calling HIGH and CRITICAL. Once the

cloud stage is added, the four strategies give similar system results (CRITICAL recall 86.2–86.4%); they differ mainly offline, when the local prediction is final. Class-balanced weights are therefore used for the deployed model. Its confusion matrix is shown in Fig. 7.3.

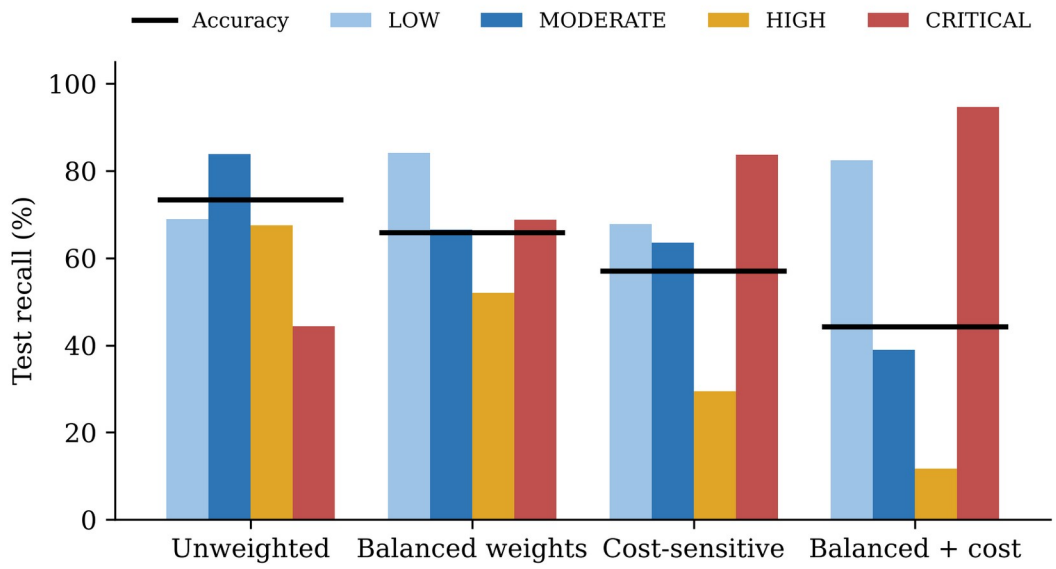


Fig. 7.3 Per-class recall (bars) and accuracy (black lines) of the selected model under four imbalance strategies.

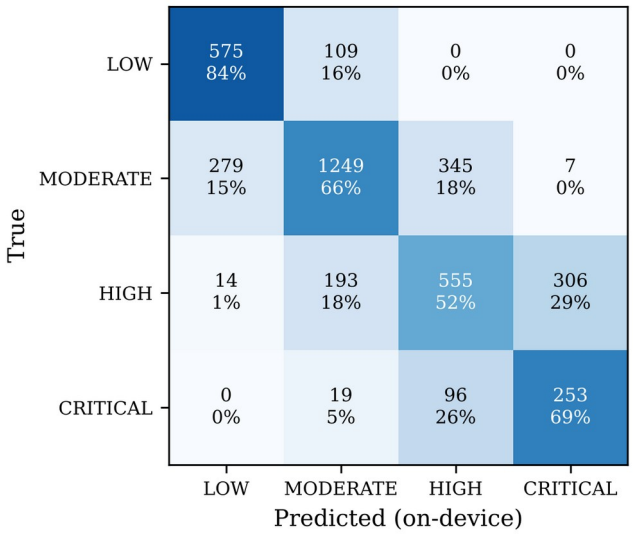


Fig. 7.4 Confusion matrix of the deployed (class-balanced) on-device model on the test set.

7.3 Deployment Benchmark

Table 7.4 ONNX Runtime benchmark, batch size 1, one CPU core

Model	Mean (µs)	p95 (µs)	p99 (µs)	Size (KB)	RAM (MB)	Work / inference
Logistic regression	6	7	17	0.7	8.5	20 MAC
Decision tree	6	8	16	18.8	9.0	8
Random forest	25	35	52	15319.8	196.1	2,180

Model	Mean (µs)	p95 (µs)	p99 (µs)	Size (KB)	RAM (MB)	Work / inference
Gradient boosting	23	34	46	442.2	14.3	2,400
MLP (32-16)	11	18	34	4.4	8.7	736 MAC

RAM: resident-memory increase after session creation. Work: multiply–accumulates (MAC) or tree-node comparisons.

The selected model is 0.66 KB and takes 6 µs on average (p99 17 µs), while the random forest needs 15.0 MB for no accuracy gain. These are CPU figures; published smartphone benchmarks [19] show far larger networks running in milliseconds, so on-device inference is not a bottleneck.

7.4 Hybrid Accuracy–Latency Trade-off

Table 7.5 Operating points on the test set (thresholds chosen on validation)

Configuration	θc	Esc. (%)	Accuracy (%)	95% CI	CRITICAL recall (%)	Total latency (s)	Reduction
Cloud-only	–	100.0	89.6	[88.6, 90.5]	87.6	12.70	–
On-device only	–	0.0	65.8	[64.4, 67.2]	68.8	5.90	53.5%
Hybrid, latency-first	0.500	7.8	69.2	[67.8, 70.5]	73.1	6.44	49.3%
Hybrid, balanced	0.775	82.5	87.7	[86.7, 88.7]	90.3	11.51	9.4%
Hybrid, safety-aware	**0.725**	**74.7**	**86.3**	**[85.2, 87.4]**	**86.4**	**10.98**	**13.6%**

Total latency adds the fixed 5.9 s of non-AI stages reported for STSMIRS. Hybrid values are simulated.

Meeting the accuracy goal (cloud-only minus 3 pp on validation) requires escalating 74.7% of events with the safety-aware policy. It reaches 86.3% accuracy and 86.4% CRITICAL recall (cloud-only: 89.6% and 87.6%) and reduces the estimated end-to-end time from 12.70 s to 10.98 s (13.6%). The latency-first point escalates only 7.8% and cuts total latency by 49.3%, but its accuracy (69.2%) is far from cloud level. Fig. 7.5 shows the full trade-off curve and Fig. 7.6 the resulting latency breakdown.

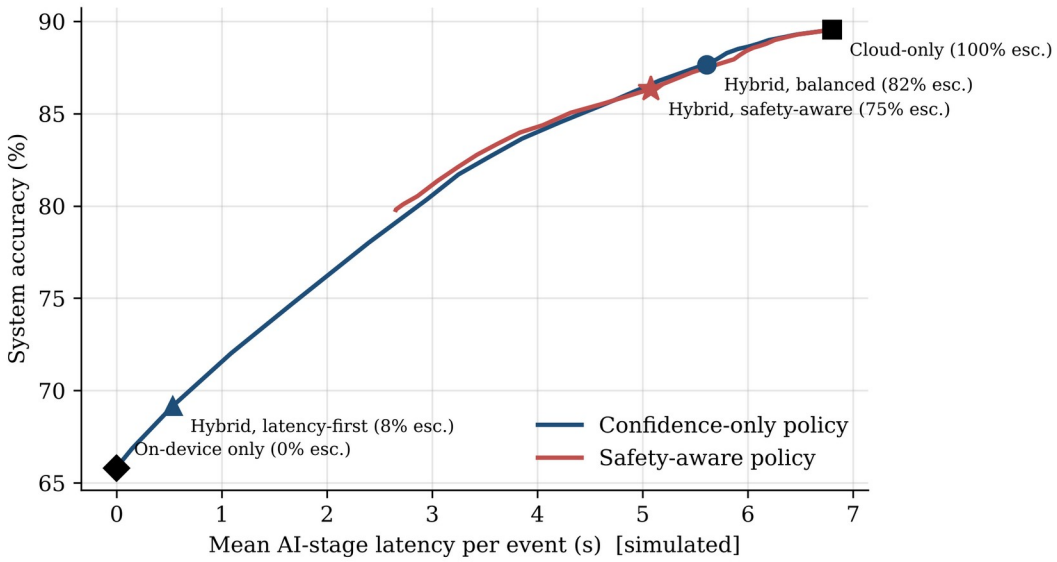


Fig. 7.5 Simulated system accuracy versus mean AI-stage latency as θc varies, with the validation-selected operating points.

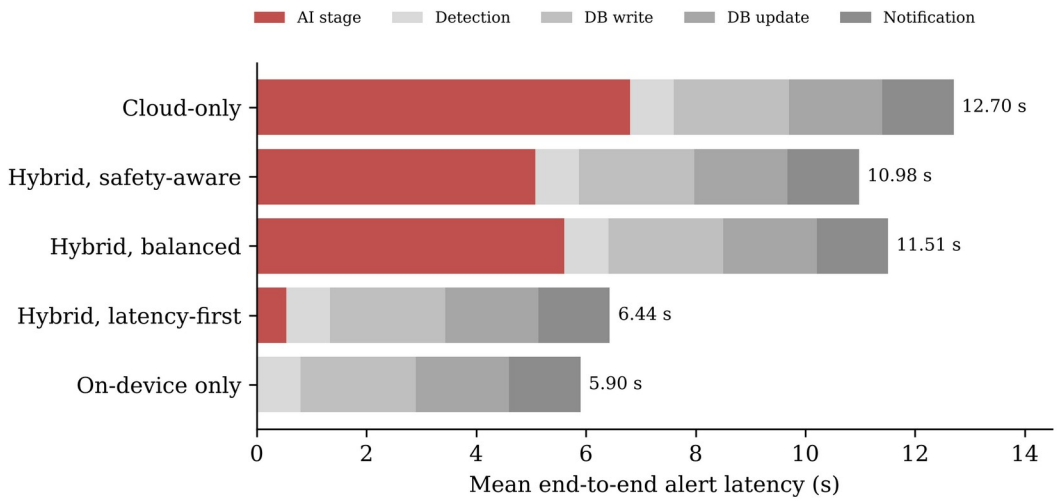


Fig. 7.6 Estimated end-to-end alert latency: AI stage (red) plus the fixed STSMIRS pipeline stages.

7.5 Sensitivity to the Cloud Model

In the proxy simulation the cloud's accuracy on escalated cases is 88.1%. Fig. 7.7 varies this directly: system accuracy and CRITICAL recall fall almost linearly with it. If the LLM were only 70% accurate on these ambiguous cases, system accuracy would be about 72.9% and CRITICAL recall about 68.6%. This is the largest uncertainty in the study, and it can only be resolved by running the real LLM on real escalated cases.

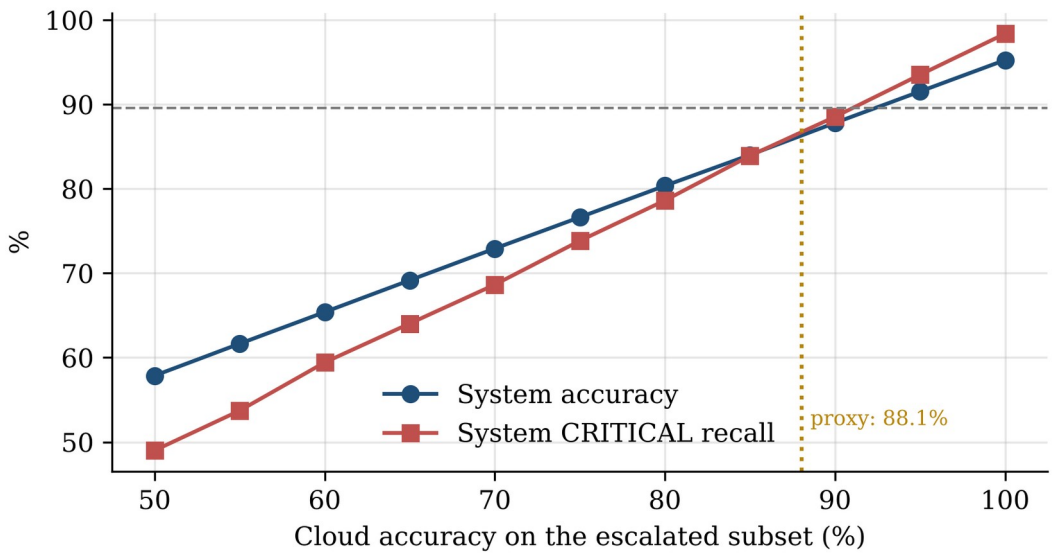


Fig. 7.7 Safety-aware operating point: system accuracy and CRITICAL recall versus the cloud model's accuracy on escalated cases.

7.6 Robustness to Distribution Shift

Table 7.6 Stress tests with model and θc fixed

Condition	Device acc. (%)	Device CRIT. rec. (%)	Escalated (%)	Hybrid acc. (%)	Hybrid CRIT. rec. (%)
In-distribution	67.2	71.2	74.2	86.3	88.5
GPS noise x3	58.8	66.0	73.7	84.2	88.5
Gazetteer error 30%	64.3	70.9	76.1	85.6	89.4
Prior shift (CRITICAL 18%)	66.6	72.2	78.0	86.3	88.7
Combined	58.3	69.3	78.2	85.3	89.8

Fresh test sets with tripled GPS error, a 30% hazard-flag error rate, a doubled CRITICAL share and their combination were evaluated with the model and threshold fixed. On-device accuracy degrades most under GPS noise, but hybrid accuracy stays within 2.1 pp of the in-distribution value because lower confidence automatically increases escalation.

7.7 Blockchain Performance

Table 7.7 Blockchain digital-ID performance (demo network, one CPU core)

Measure	Value
Authority nodes	Tourism Department, City Police, Immigration Office
Blocks / transactions	26 / 221
Registration (sign + record), mean	0.21 ms
Verification (5 checks), mean	0.17 ms
Chain file size (200 tourists + incidents)	177 KB

Registering a tourist takes about 0.2 ms and verifying a credential about 0.2 ms, so the blockchain layer adds negligible time compared with the seconds spent in the alert pipeline. Because only hashes are stored, the chain for 200 tourists and their incidents is about 177 KB.

7.8 Discussion

The evidence supports four conclusions: a sub-kilobyte on-device model can make the Stage-2 decision in microseconds; on these features, model family matters little and a simple, well-calibrated model suffices; class-balanced training substantially improves offline CRITICAL recall; and selective escalation offers an operator-tunable trade-off between latency and accuracy. In practice θ_c can be set per zone: latency-first in low-risk urban areas with good connectivity, safety-aware in designated high-risk zones, and in remote areas with poor coverage the offline CRITICAL recall of the device model becomes the deciding factor.

7.9 Limitations

- All data are synthetic; the absolute numbers show relative trade-offs, not expected field performance.
- The cloud LLM was not executed; it is represented by a calibrated proxy and bounded by the sensitivity analysis.
- Latency and memory were measured on a CPU, not on smartphones, and energy was not measured.
- End-to-end latency adds simulated AI-stage time to the stage times reported for STSMIRS; network variability and queueing are not modelled.
- GPS spoofing and the retraining loop are not evaluated.
- The blockchain runs as a local permissioned network; the demo keeps all authority keys in one file, whereas a deployment would give each organisation its own node and key store. The smart contract was tested on a local Ethereum chain, not on a public network.

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

This project built a Smart Tourist Safety Monitoring and Incident Response System that combines geo-fencing, AI risk classification and a blockchain-based digital ID. A permissioned proof-of-authority blockchain issues signed digital tourist IDs that police can verify by QR code, rejects forged, expired and revoked IDs, keeps personal data off-chain, and records every incident alert as a tamper-evident audit trail; an equivalent Solidity smart contract passed all its tests. For the AI part, the project addressed the cloud-LLM latency bottleneck of STSMIRS with a hybrid edge–cloud design. A geofencing layer (Haversine distance, circular and polygon zones, route deviation and idle detection) feeds an on-device classifier. Among eight candidates, model families proved statistically similar, so a 0.66 KB logistic-regression model running in about 6 µs was selected, and class-balanced training raised its standalone CRITICAL recall from 44.3% to 68.8%. In simulation, the safety-aware hybrid kept accuracy within about 3 pp of the cloud-only baseline (86.3% vs 89.6%) while reducing estimated end-to-end latency by 13.6%; reductions of up to 49.3% are possible at lower accuracy. The integrated system was validated end to end and passed all 130 automated tests, including end-to-end browser tests of the deployed tourist and police applications. The attainable benefit depends critically on the LLM's accuracy on the ambiguous cases it receives.

8.2 Future Enhancements

1. Evaluate on real, labelled tourist trajectories, such as an anonymised STSMIRS deployment log.
2. Replace the cloud stand-in with the production LLM and measure its real accuracy and latency on escalated cases.
3. Embed the model in the Flutter app and benchmark latency, memory and energy on low- and mid-range Android and iOS phones.
4. Learn the escalation (deferral) policy instead of using a fixed threshold.
5. Implement the retraining loop from cloud-verified labels and add GPS-spoofing detection.
6. Download offline map tiles for each tourist's itinerary area (leaflet.offline or MBTiles with flutter_map), so the map is shown even without internet, and add an SMS fallback for sending alerts on weak 2G signal.

7. Deploy the smart contract on a multi-organisation permissioned network (Hyperledger Besu or Polygon) and integrate Aadhaar e-KYC and a mobile identity wallet.

REFERENCES

[1] E. Ragavan, S. Sanjay, and S. Arulmurugan, “Smart tourist monitoring incident system using geo-fencing and blockchain-based digital ID,” in Proc. 2nd Int. Conf. Computing for Sustainability and Intelligent Future (COMP-SIF), 2026, doi: 10.1109/COMP-SIF69752.2026.11481856.

[2] I. Sasaki, M. Arikawa, M. Lu, T. Utsumi, and R. Sato, “Data-driven geofencing design for point-of-interest notifiers utilizing genetic algorithm,” ISPRS Int. J. Geo-Inf., vol. 13, no. 6, Art. no. 174, 2024.

[3] V. Karuppuchamy, S. Hari Haran, A. Pravinkumar, and S. Gokulnath, “An LSTM-based predictive geo-fencing framework for intelligent tourist safety monitoring,” in Proc. 6th Int. Conf. ICTMIM, 2026.

[4] Y. Zheng, “Trajectory data mining: An overview,” ACM Trans. Intell. Syst. Technol., vol. 6, no. 3, Art. no. 29, 2015.

[5] Y. Zheng, X. Xie, and W.-Y. Ma, “GeoLife: A collaborative social networking service among user, location and trajectory,” IEEE Data Eng. Bull., vol. 33, no. 2, pp. 32–39, 2010.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” IEEE Internet Things J., vol. 3, no. 5, pp. 637–646, 2016.

[7] Y. Kang et al., “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in Proc. ASPLOS, 2017, pp. 615–629.

[8] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” in Proc. ICPR, 2016, pp. 2464–2469.

[9] ONNX Runtime Developers, “ONNX Runtime.” [Online]. Available: https://onnxruntime.ai/

[10] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[11] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[12] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016, pp. 785–794.

[13] C. K. Chow, “On optimum recognition error and reject tradeoff,” IEEE Trans. Inf. Theory, vol. 16, no. 1, pp. 41–46, 1970.

[14] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in Proc. NeurIPS, 2017, pp. 4878–4887.

[15] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in Proc. IEEE CVPR, 2001, pp. I-511–I-518.

[16] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” arXiv:2305.05176, 2023.

[17] H. He and E. A. Garcia, “Learning from imbalanced data,” IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, 2009.

[18] C. Elkan, “The foundations of cost-sensitive learning,” in Proc. IJCAI, 2001, pp. 973–978.

[19] A. Ignatov et al., “AI Benchmark: Running deep neural networks on Android smartphones,” in Proc. ECCV Workshops, 2018, pp. 288–314.

[20] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[21] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[22] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” Ethereum Yellow Paper, 2014.

[23] A. Mühle, A. Grüner, T. Gayvoronskaya, and C. Meinel, “A survey on essential components of a self-sovereign identity,” Computer Science Review, vol. 30, pp. 80–86, 2018.

[24] E. Androulaki et al., “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in Proc. EuroSys, 2018, pp. 1–15.

[25] R. C. Merkle, “A digital signature based on a conventional encryption function,” in Proc. CRYPTO ’87, 1988, pp. 369–378.

[26] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” J. Cryptographic Engineering, vol. 2, no. 2, pp. 77–89, 2012.

APPENDIX A: APPLICATION SCREENSHOTS

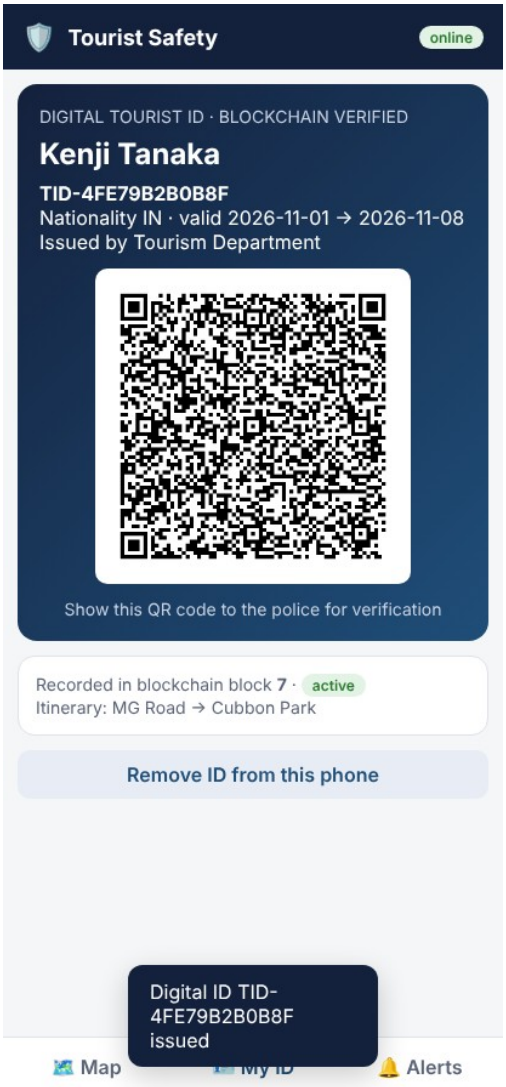


Fig. A.1 Tourist app: blockchain digital ID with QR credential.

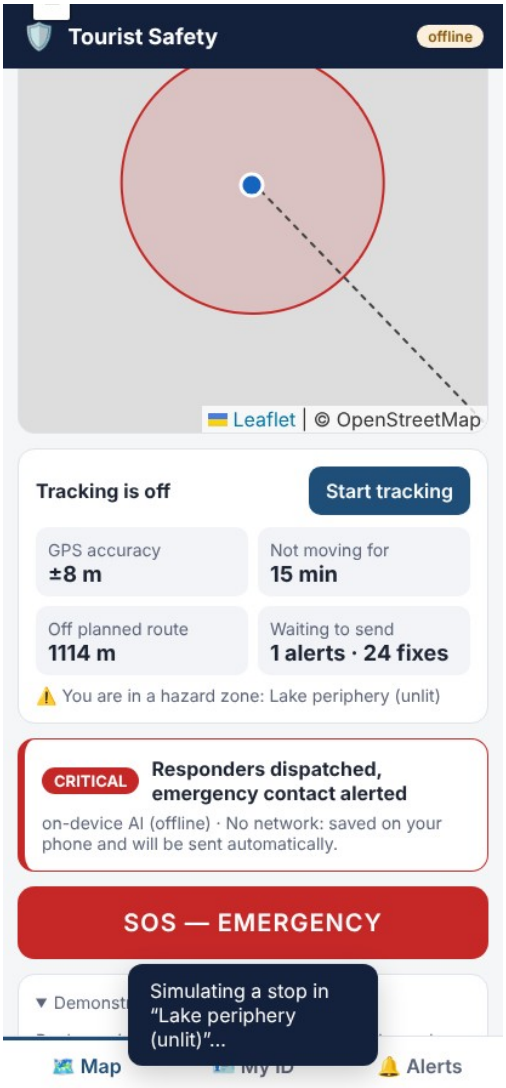


Fig. A.2 Tourist app without network: alert decided on the phone and saved for later delivery.

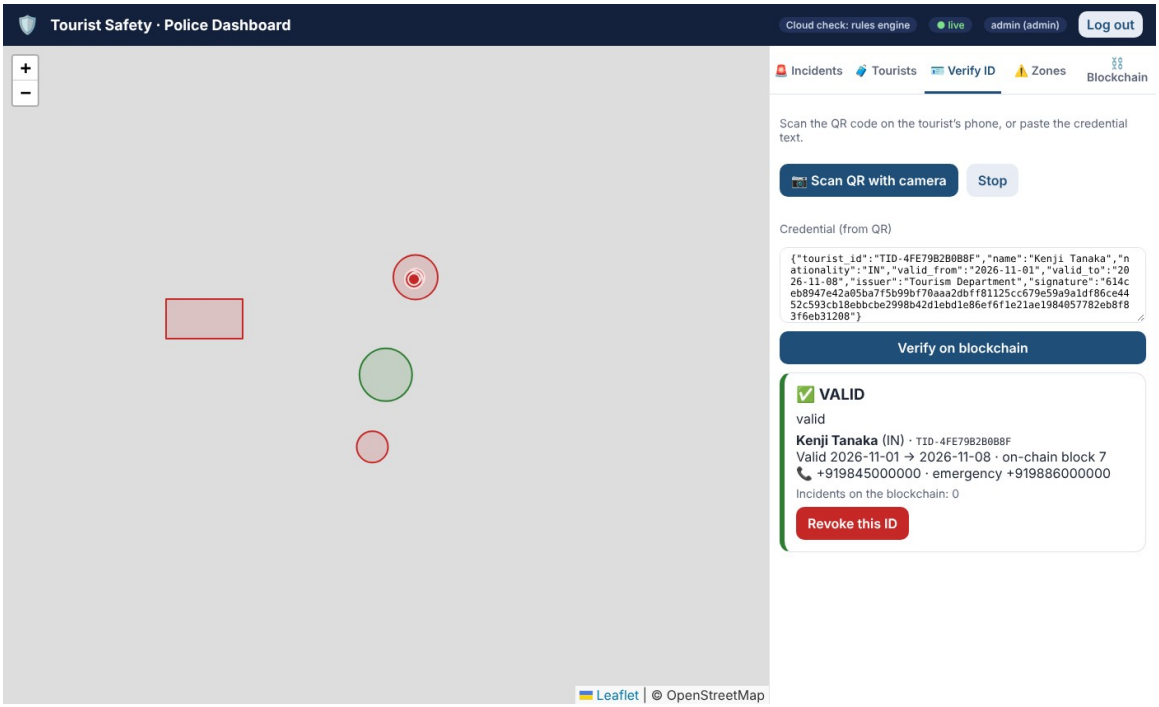


Fig. A.3 Police dashboard: digital-ID verification.

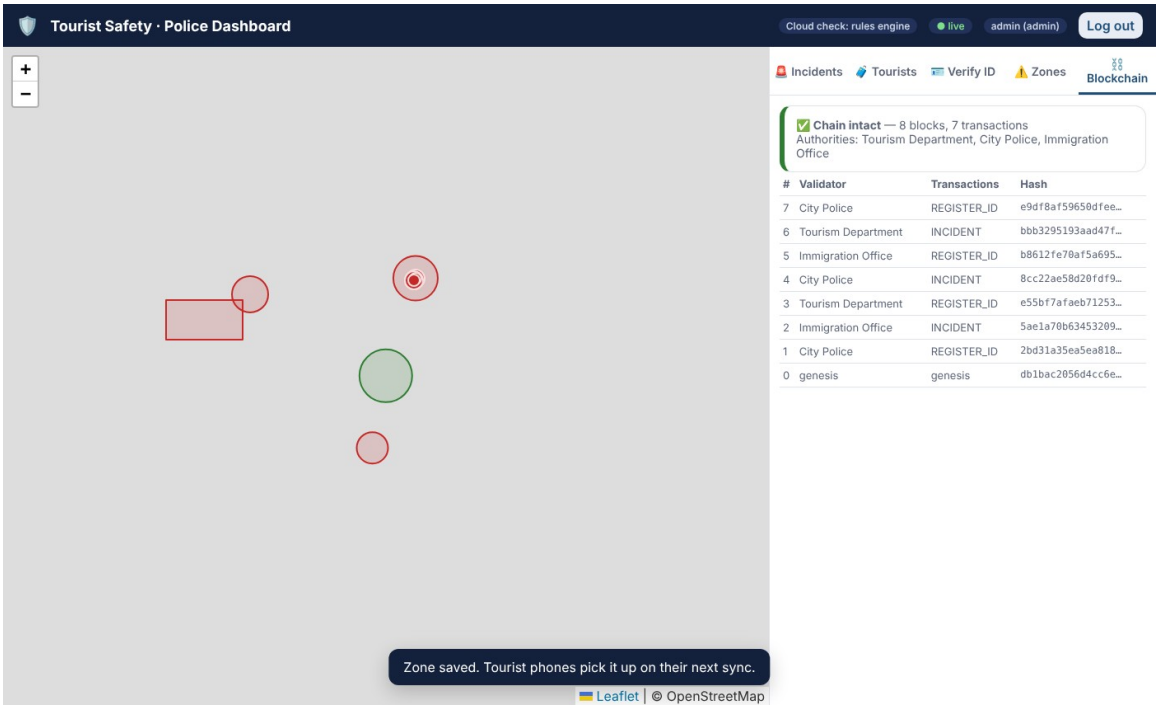


Fig. A.4 Police dashboard: blockchain explorer after the end-to-end tests.

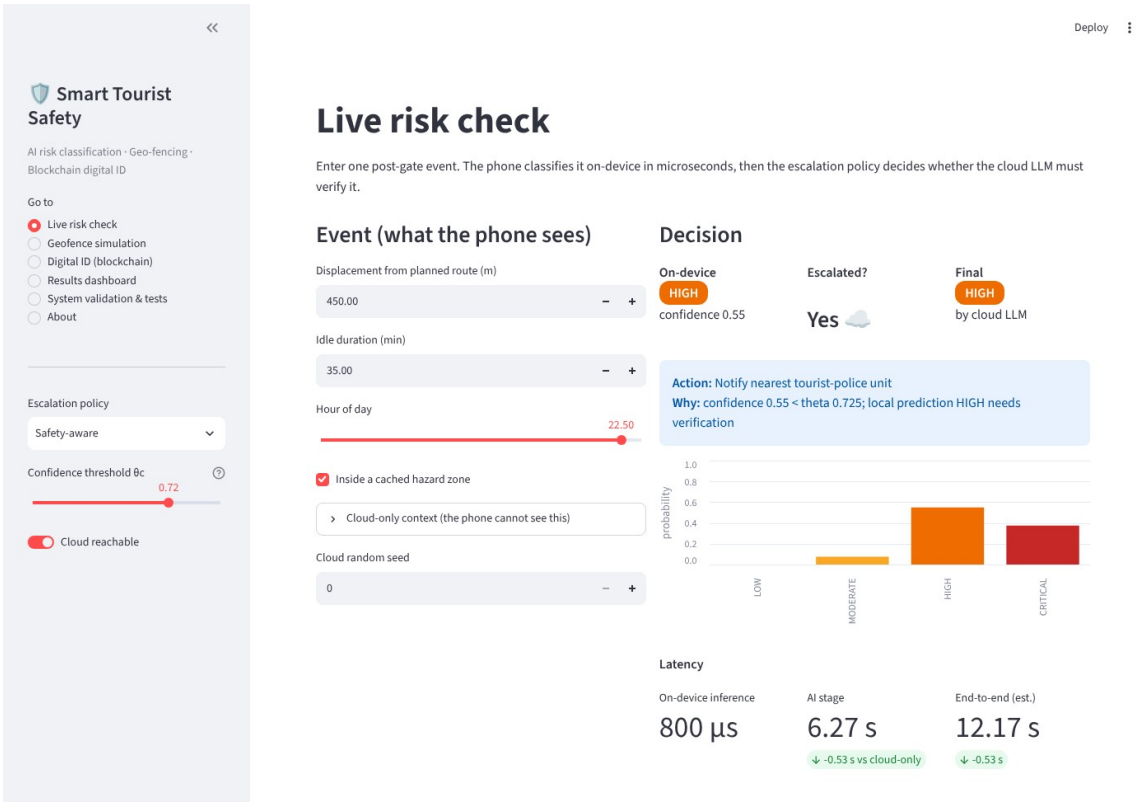


Fig. A.5 Analytics dashboard — live risk check: on-device probabilities, escalation decision, final class, action and latency.

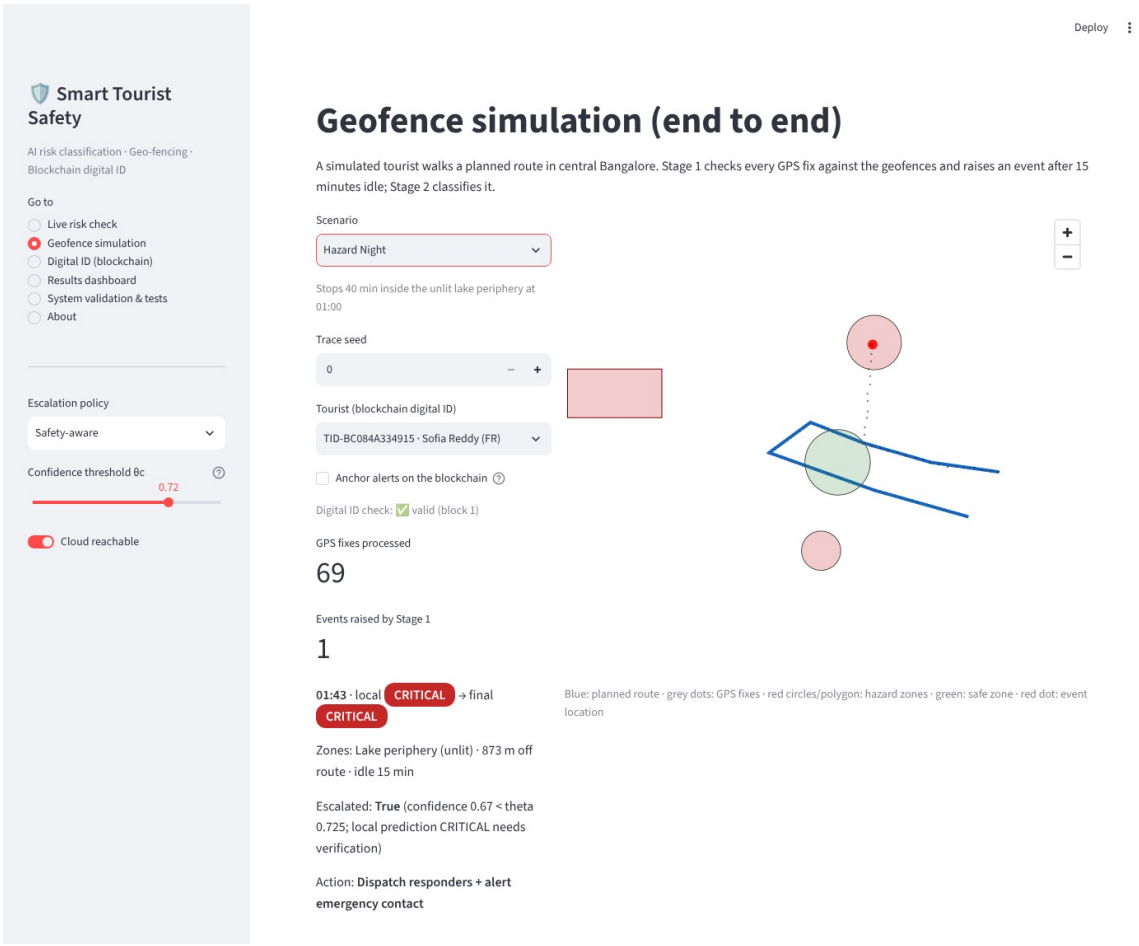


Fig. A.6 Geofence simulation: night-time stop inside the unlit lake periphery, raised by Stage 1 and escalated to the cloud.

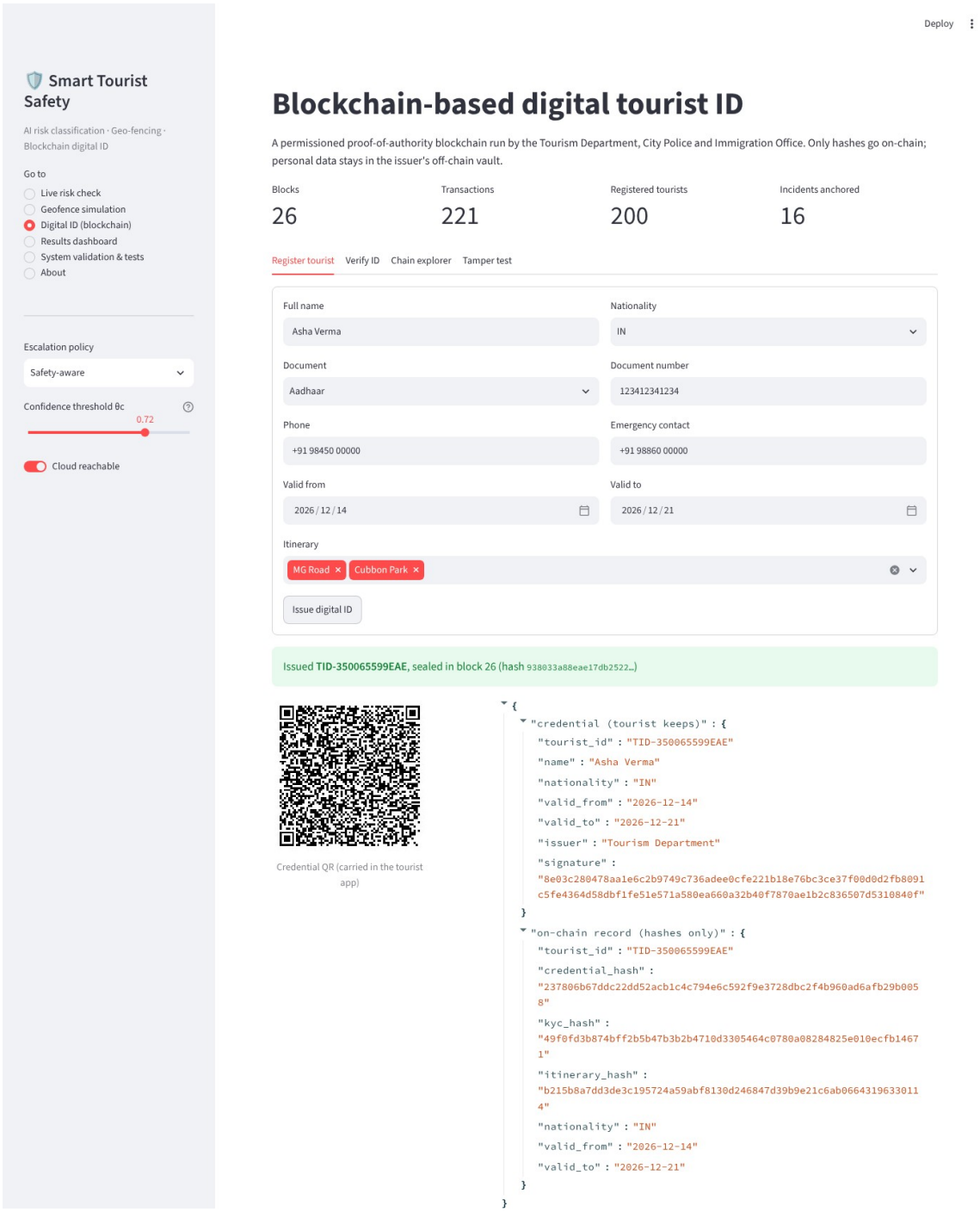


Fig. A.7 Digital ID page: issuing a blockchain digital ID with its QR credential and on-chain record (hashes only).

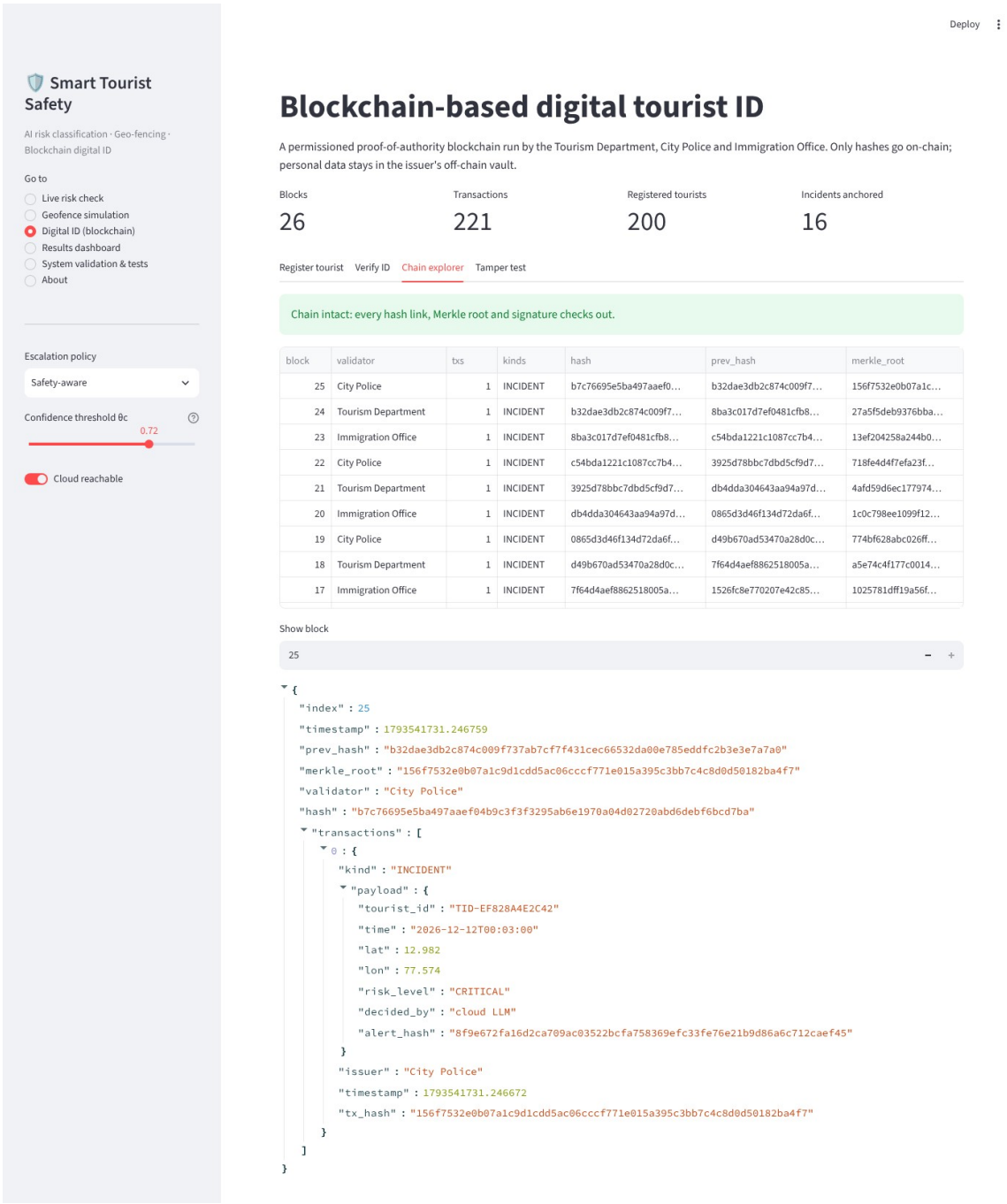
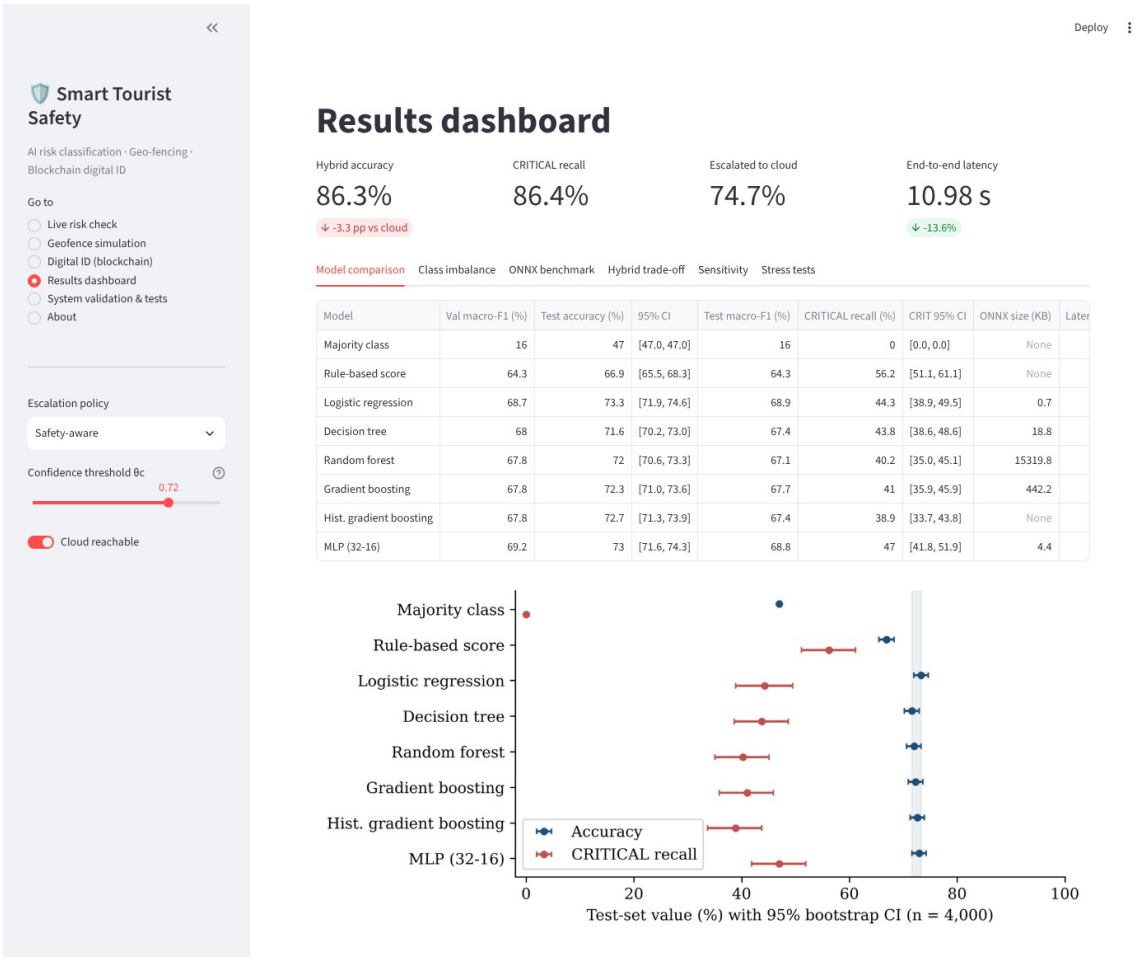


Fig. A.8 Chain explorer: integrity check and the latest blocks.



Model	Val macro-F1 (%)	Test accuracy (%)	95% CI	Test macro-F1 (%)	CRITICAL recall (%)	CRIT 95% CI	ONNX size (KB)	Later
Majority class	16	47	[47.0, 47.0]	16	0	[0.0, 0.0]	None	
Rule-based score	64.3	66.9	[65.5, 68.3]	64.3	56.2	[51.1, 61.1]	None	
Logistic regression	68.7	73.3	[71.9, 74.6]	68.9	44.3	[38.9, 49.5]	0.7	
Decision tree	68	71.6	[70.2, 73.0]	67.4	43.8	[38.6, 48.6]	18.8	
Random forest	67.8	72	[70.6, 73.3]	67.1	40.2	[35.0, 45.1]	15319.8	
Gradient boosting	67.8	72.3	[71.0, 73.6]	67.7	41	[35.9, 45.9]	442.2	
Hist. gradient boosting	67.8	72.7	[71.3, 73.9]	67.4	38.9	[33.7, 43.8]	None	
MLP (32-16)	69.2	73	[71.6, 74.3]	68.8	47	[41.8, 51.9]	4.4	

Fig. A.9 Results dashboard.

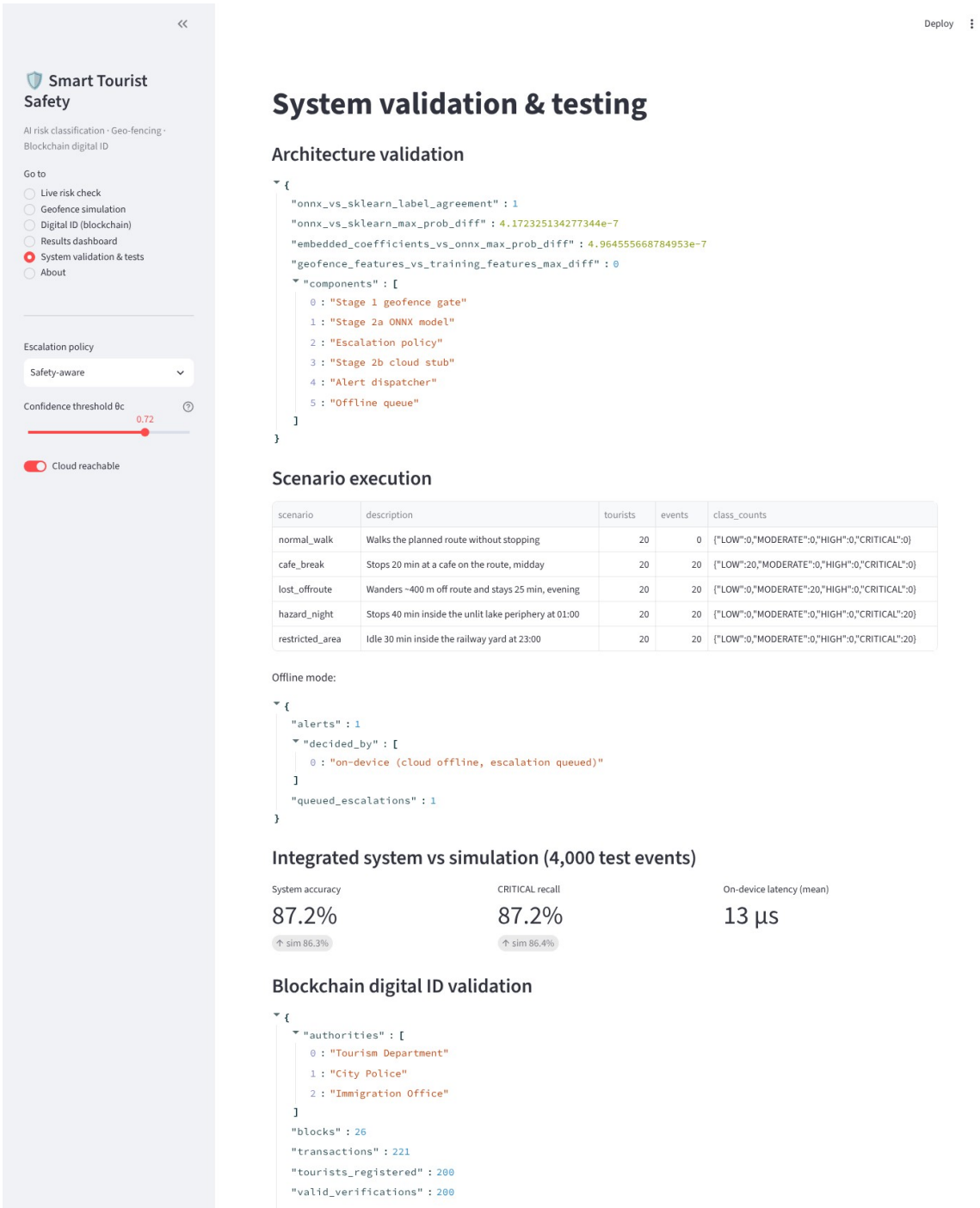


Fig. A.10 System validation and automated test results.

APPENDIX B: HOW TO RUN THE PROJECT

```
pip install -r requirements.txt

streamlit run app/streamlit_app.py      # demo (trained models are included)

python run_all.py                       # full study: data, training, ONNX,
simulation
python make_figures.py                  # all figures
python validate_system.py               # architecture validation + end-to-end
execution
python run_server.py --https           # the real application (tourist app +
police dashboard)

python run_tests.py                     # all tests -> results/test_report.html

cd contracts && npm install && npm test # smart-contract tests (Node.js 18+)
```